

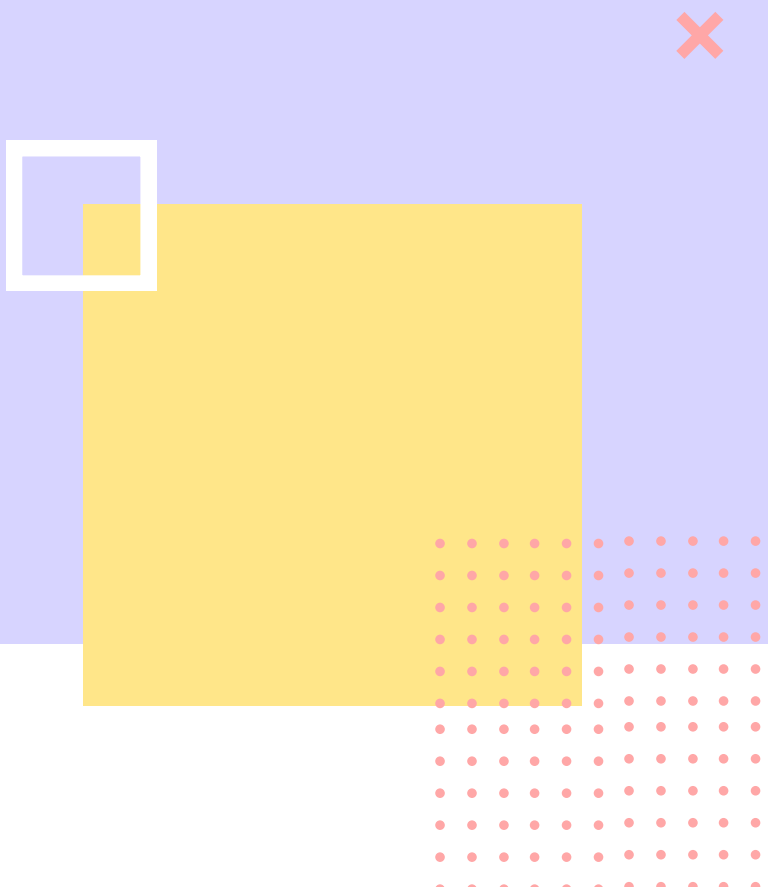


# DirectX 2D PORTFOLIO

기술문서

임시온

# CONTENTS



## 1 게임 개요

- 게임 소개 및 개발 정보

## 3 Editing 기능

- Editing 모드 → Game 모드 변환 / 선행보간

## 5 UI 및 기타 기능

- UIManager, UIPanel, 튜토리얼 기능, 클리어 정보 저장/ Shader & ValueBuffer

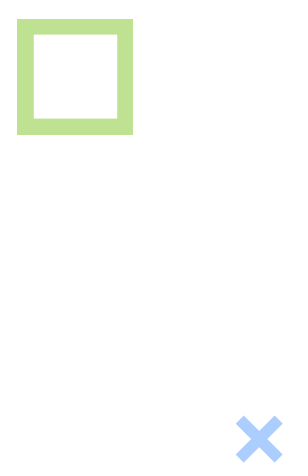
## 2 Data Table & 맵 로드

- SpiderGame, EditTileObject / 인스턴싱

## 4 SpiderObject

- SpiderPhysics, Player & Enemy, EnemyManager, SpiderSilk, 충돌처리

## 6 Review

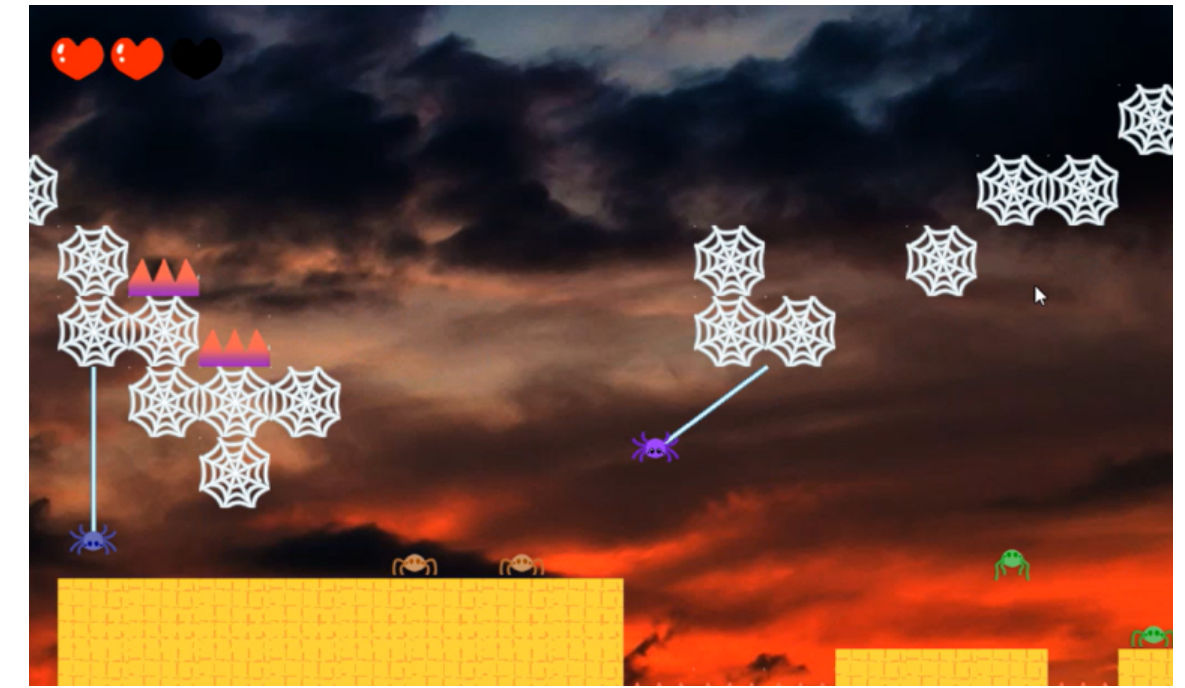
- 개발 후기
- 

# 게임 개요

2D 플랫폼머 아케이드 창작 게임 <I'm Spider>

-> 거미가 거미줄을 쏘며 이동 / 장애물과 적을 피해 골인하는 게임

|           |                                                                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------------------------------|
| 제목        | I'm Spider                                                                                                                    |
| 장르        | 2D 플랫폼머, 아케이드, 퍼즐                                                                                                             |
| 개발 환경     | Visual Studio 2022                                                                                                            |
| 개발 언어     | C++, HLSL                                                                                                                     |
| 개발 기간     | 2024.12.09. ~ 2024.12.23. (15일)                                                                                               |
| 라이브러리     | DirectX 11, STL, TinyXML, FMOD, ImGui                                                                                         |
| 실행 영상     | <a href="https://youtu.be/gVfd2UThpls">https://youtu.be/gVfd2UThpls</a>                                                       |
| GitHub 주소 | <a href="https://github.com/shieon51/DX2DPortfolio1_ImSpider.git">https://github.com/shieon51/DX2DPortfolio1_ImSpider.git</a> |



# DataTable & 맵 로드

## DataTables에서 특정 Stage 맵 구성 정보 불러오기

게임 진행을 관리하는 SpiderGame 클래스에서 LoadStage 메소드를 통해 .csv 파일로 된 각 스테이지 타일 정보를 지정된 타일 타입에 따라 구별

| G  | H  | I | J | K | L | M | N | O | P  | Q  | R | S | T | U | V | W | X | Y | Z    | AA | AB   | AC |
|----|----|---|---|---|---|---|---|---|----|----|---|---|---|---|---|---|---|---|------|----|------|----|
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 10 | 12 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 2  | 2    | 2  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 2221 | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0  | 0  | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1    | 1  | 1    | 1  |
| 1  | 1  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0  | 0  | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 2121 | 1  | 1    | 1  |
| 11 | 11 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 0  | 0  | 0 | 0 | 0 | 0 | 2 | 2 | 2 | 2 | 2    | 2  | 2    | 2  |



타일 타입(TileType)은  
이미지 왼쪽 상단부터 차례대로  
0, 1, 2, 3, 10, 11, 12,  
20, 21, 22, 30

```
public:
    enum DeadObjectType
    {
        OBSTACLE = 1,
        ENEMY = 2
    };

    enum TileType
    {
        //빈 공간
        CAN_EDIT = 0,
        NO_EDIT = 1,

        //일반지형
        RAGULAR = 2,

        //골인지점
        GOAL = 3,

        //장애물: 십의 자리 1 -> DeadObjectType 1
        DOWN_THORN = 10,
        UP_THORN = 11,
        COBWEB = 12,

        //적: 천의 자리 2 -> DeadObjectType 2
        JUMP_ENEMY = 20,
        MOVE_ENEMY = 21,
        HANG_ENEMY = 22,

        //플레이어
        PLAYER = 30
    };
```

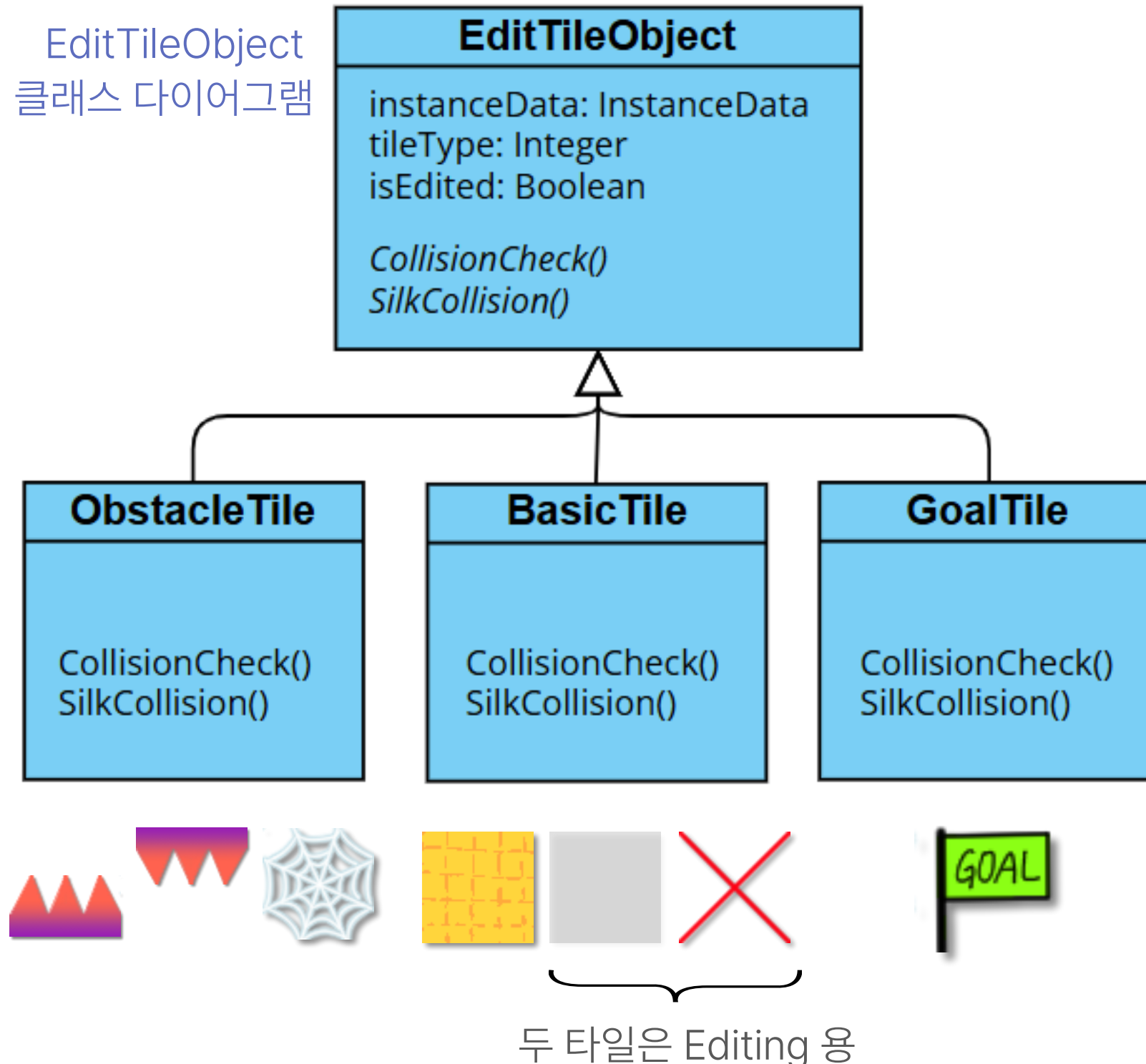
SpiderGame 클래스에 enum으로  
TileType과 DeadObjectType을 정의



# 인스턴싱을 이용하여 타일(EditTileObject)배치

SpiderGame의 CreateTiles 메소드로 구현

EditTileObject  
클래스 다이어그램



```
for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        int index = y * width + x;
        EditTileObject* editTile;

        /* 타입에 따라 나누기 - 일반, 장애물, 기타(에디팅용)
        if (tileTypes[index] / 10 == DeadObjectType::OBSTACLE) // 장애물
            editTile = new ObstacleTile(tileSize, &instances[index]);
        else if (tileTypes[index] == TileType::RAGULAR) // 일반 지형
            editTile = new BasicTile(tileSize, &instances[index]);
        else if (tileTypes[index] == TileType::GOAL) // 골인
            editTile = new GoalTile(tileSize, &instances[index]);
        else // 그 외 (적 타일, 플레이어 타일)
            editTile = new BasicTile(tileSize, &instances[index]);

        Vector2 curTilePos = tileSize * Vector2(x, -y + height - 1) - Vector2(10, 10);

        editTile->SetPos(curTilePos);
        editTile->SetParent(this);
        editTile->UpdateWorld();
        editTile->SetPlayer(player);
        editTile->SetSilkCollider(player->GetSpiderSilk()->GetSilkCollider());
        editTile->SetTileType(tileTypes[index]); // 타일 타입 넘겨주기 -> 타입에 따라 에디팅 가
        instances[index].transform = XMMatrixTranspose(editTile->GetWorld());
        instances[index].curFrame = ConvertTypeNumToFrame(index, editTile->GetGlobalPos());
        instances[index].maxFrame = maxFrame;
        editTileObjects.push_back(editTile);
    }
}

instanceBuffer = new VertexBuffer(instances.data(), sizeof(InstanceData), size);
```

CreateTiles 메소드 일부

CreateTiles 메소드에서는 각 타일들의 부모 클래스인 **EditTileObject**를 vector에 담고, 이미지 출력은 인스턴싱을 이용하여 최적화 진행

# TileType에 따라 표시할 이미지 프레임 변환

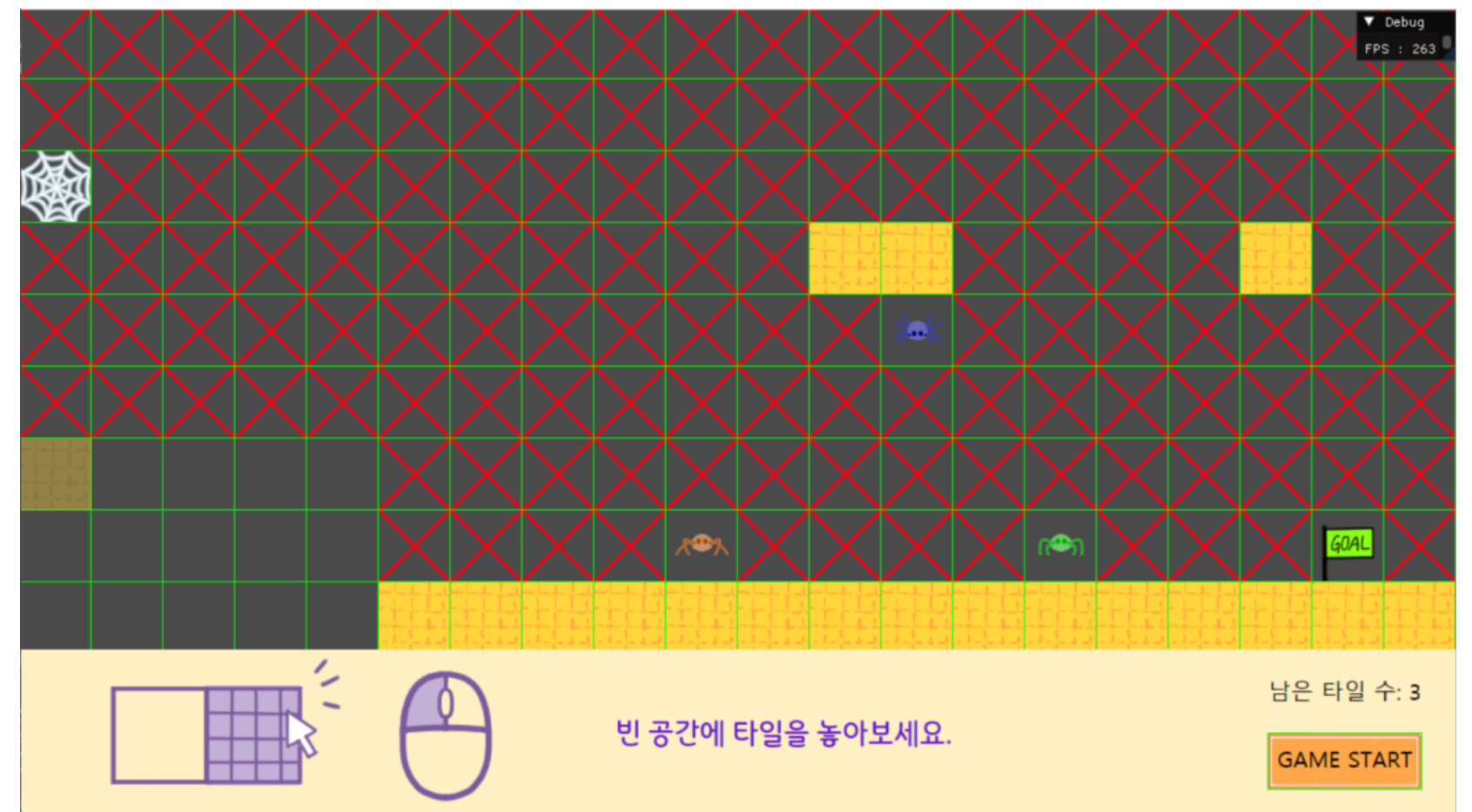
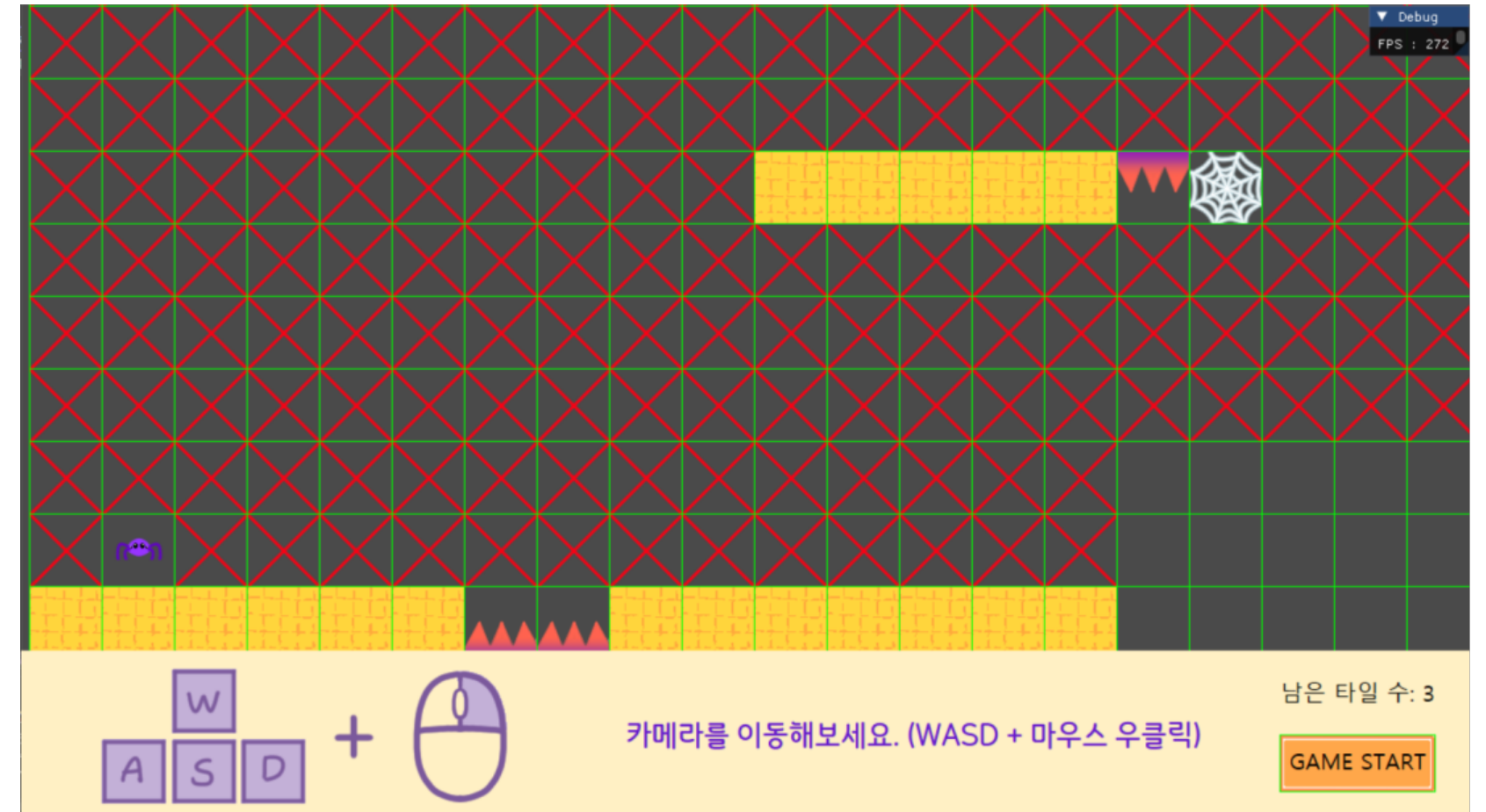
중간에 플레이어, 적과 같은 캐릭터 객체가 있는 경우  
플레이어는 초기 위치를 설정해주고,  
적은 모든 Enemy 객체의 행동과 충돌처리를 관리하는  
EnemyManager에 해당 객체를 추가 후 초기 위치를 설정

```
Float2 SpiderGame::ConvertTypeNumToFrame(const int& index, const Vector2& curTilePos)
{
    Float2 frame;

    int tileType = tileTypes[index];

    if (tileType <= GOAL) //빈 공간(수정가능/불가능), 일반지형, 골인
    {
        frame = { (float)tileType, 0 };
    }
    else if (tileType / 10 == DeadObjectType::OBSTACLE) //장애물
    {
        frame = { (float)tileType - 6, 0 };
    }
    else if (tileType / 1000 == DeadObjectType::ENEMY) //적: 앞 두자리는 적종류, 뒤 두자
    {
        frame = { (float)(tileType / 100 % 10), 1 };
        EnemyManager::Get()->AddSpiderEnemy(curTilePos, tileType); //적 만들고 푸시
    }
    else if (tileType == PLAYER) //플레이어
    {
        frame = { (float)(tileType / 10), 1 };
        player->SetPlayerInitialPos(curTilePos);
        player->SetPos(curTilePos); //초기 위치 설정
        player->SetLastLandCollisionTilePos(curTilePos);
    }
    else //예외
        frame = { 0, 0 };
}
```

ConvertTypeNumToFrame 메소드 (CreateTiles 메소드 내에서 호출)



맵 전체 출력 결과



# Editing 기능

## 게임 시작 전 Editing 가능 지역에 타일 설치 (퍼즐 요소)

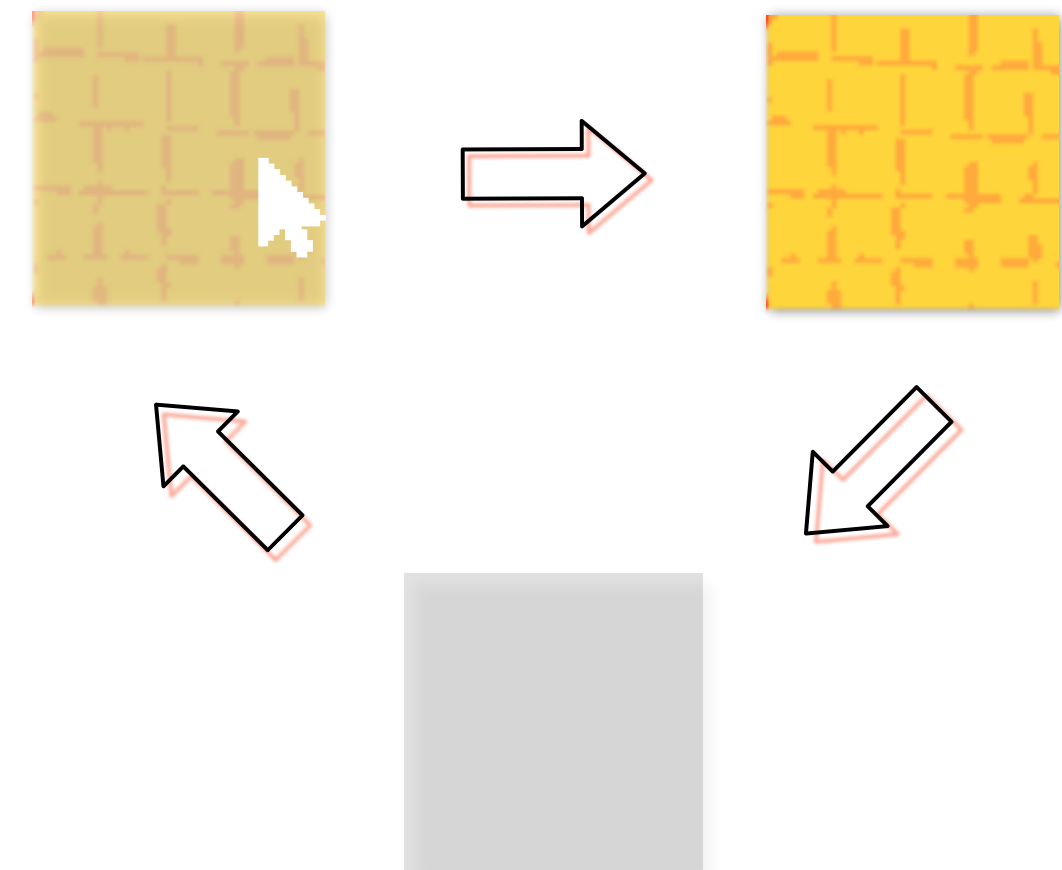
게임 시작 전 플레이어가 맵을 살펴보고 해당 스테이지를 통과하기 위해 직접 지형 타일을 일부 추가할 수 있음. (개수는 최대 3개로 제한)



Editing가능 지역에 마우스를 올리면 해당 위치에 반투명 타일 표시.  
클릭 시 해당 타일의 프레임을 변경하고 해당 타일이 수정되었음을 저장.

마우스가 Editing 가능 지역을  
가리킬 때 (반투명 타일 표시)

클릭시 지형 설치되며  
불투명 타일로 변경

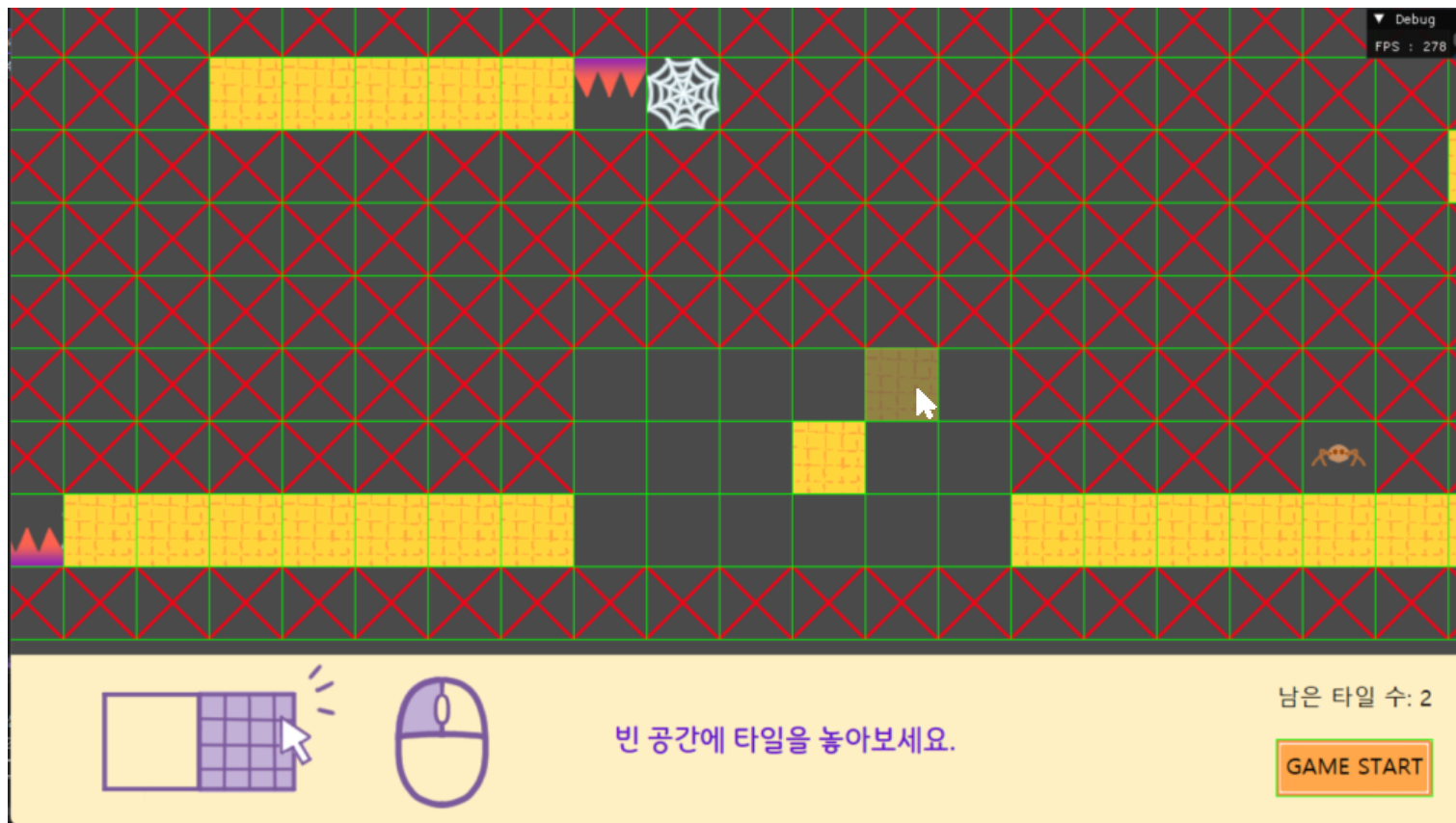


수정된 해당 지형을 재클릭 시  
타일 설치가 취소되며  
설치 전 모습(빈 타일)으로 돌아감

```
void SpiderGame::Update()
{
    if (isEditing) // 에디팅 중일 때
    {
        for (EditTileObject* editTile : editTileObjects)
        {
            EditingCollision(editTile);
        }
    }
    else //게임 중일 때
    {

```

Editing 상태인지 아닌지에 따라 충돌 처리를 달리 구현.  
→ Editing 시에는 EditingCollision 메소드를 호출



```

{
    if (editTile->IsPointCollision(CAM->ScreenToWorld(mousePos))) //현재 카메라뷰 위치에
    {
        if (editTile->GetTileType() == CAN_EDIT)
        {
            if (KEY->Up(VK_LBUTTON))
            {
                // (UI가 앞에 떠 있는 곳 클릭하면 -> 뒤에 클릭 반영 안 되도록 예외처리)
                if (UIManager::Get()->IsContain("Tutorial", mousePos)) return;

                if (editedTileNum < MAX_EDITTILE_NUM //설치 가능 타일수 제한
                    && editTile->GetIsEdited() == false) //아직 수정되지 않은 타일이면
                {
                    editTile->GetInstanceData()->curFrame = selectFrame;
                    editTile->SetIsEdited(true);
                    editedTileNum++;
                }
                else if (editTile->GetIsEdited() == true) //이미 수정된 적 있는 타일이면 취소
                {
                    editTile->GetInstanceData()->curFrame = { 0, 0 };
                    editTile->SetIsEdited(false);
                    editedTileNum--;
                }

                UpdateInstanceData();
            }
            else //마우스만 올려놓은 상태라면 -> 현재 가리키고 있는 타일 표시하기
            {
                curserPointTile->SetPos(editTile->GetGlobalPos());
                curserPointTile->UpdateWorld();
            }
        }
        else //에디팅 가능 지역이 아니라면

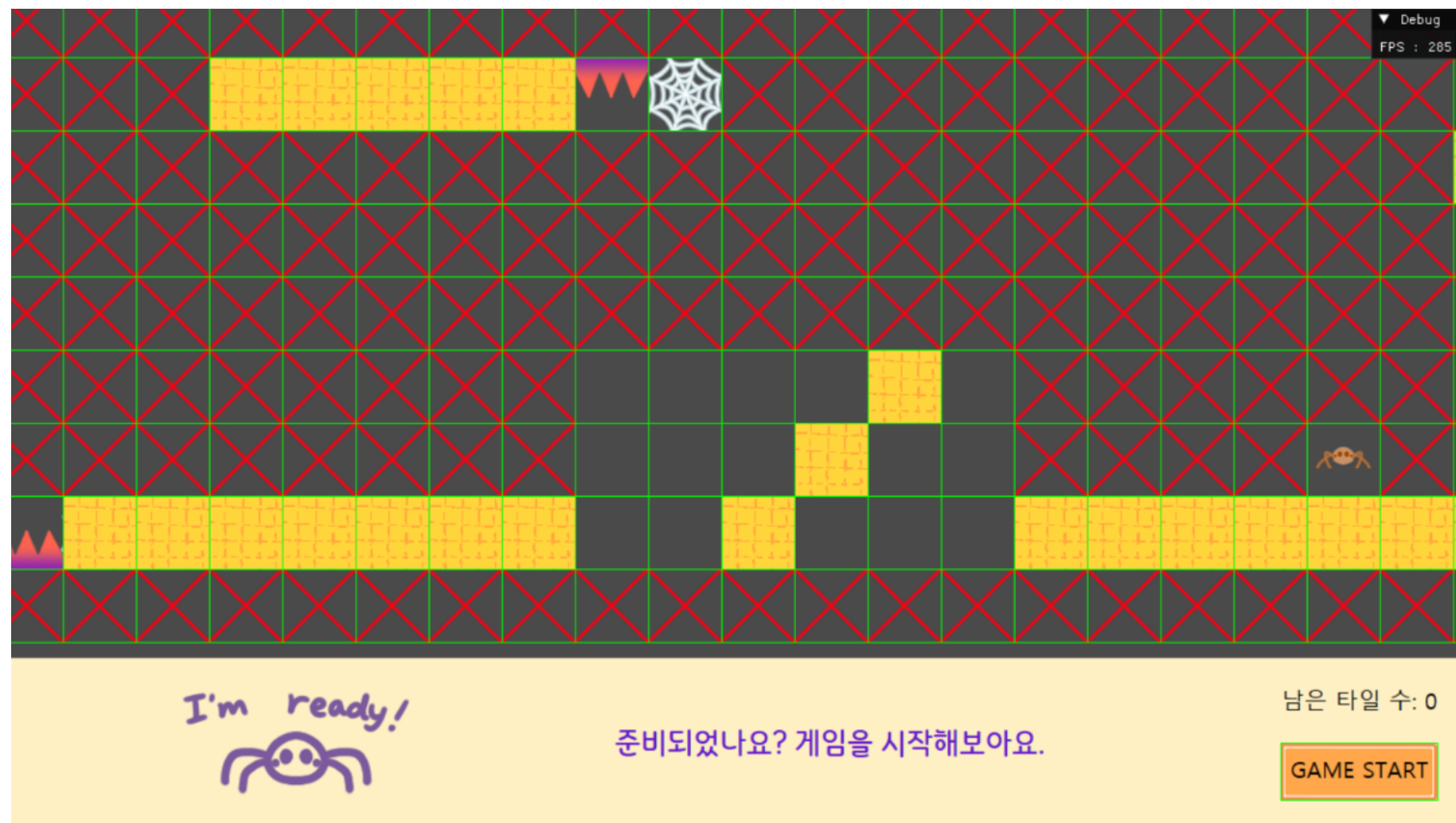
```

EditingCollision 메소드 일부

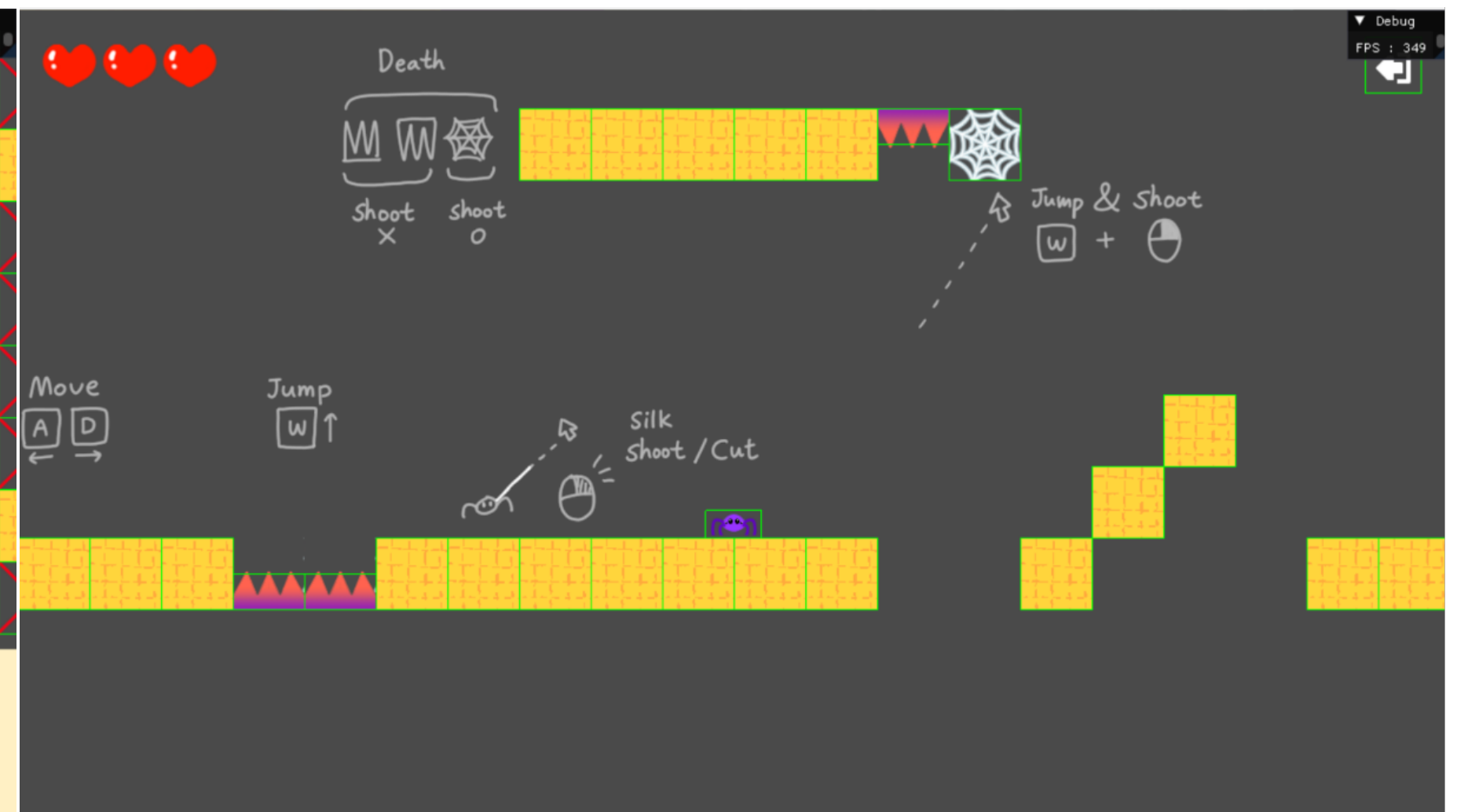


## 게임 시작 버튼 클릭 시 Editing 모드에서 Game 모드로 전환

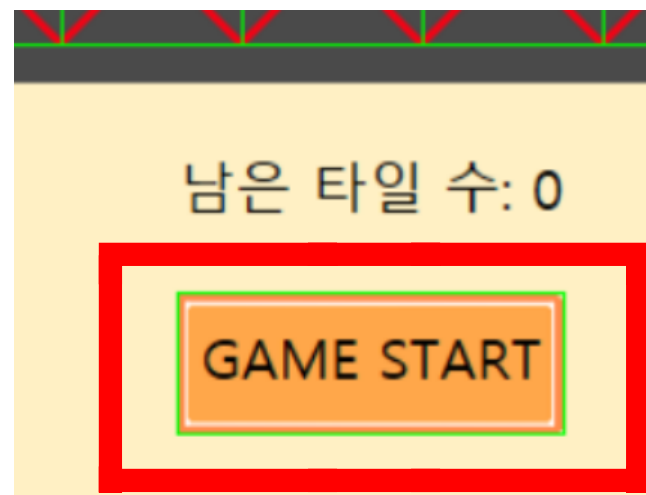
버튼 클릭 시 StartGame 메소드를 호출하면 현재 맵 정보와 수정사항을 반영해 Game 모드에서만 쓰이는 타일들(일반타일, 장애물)만 남기고, 나머지는 모두 빈 타일로 변경



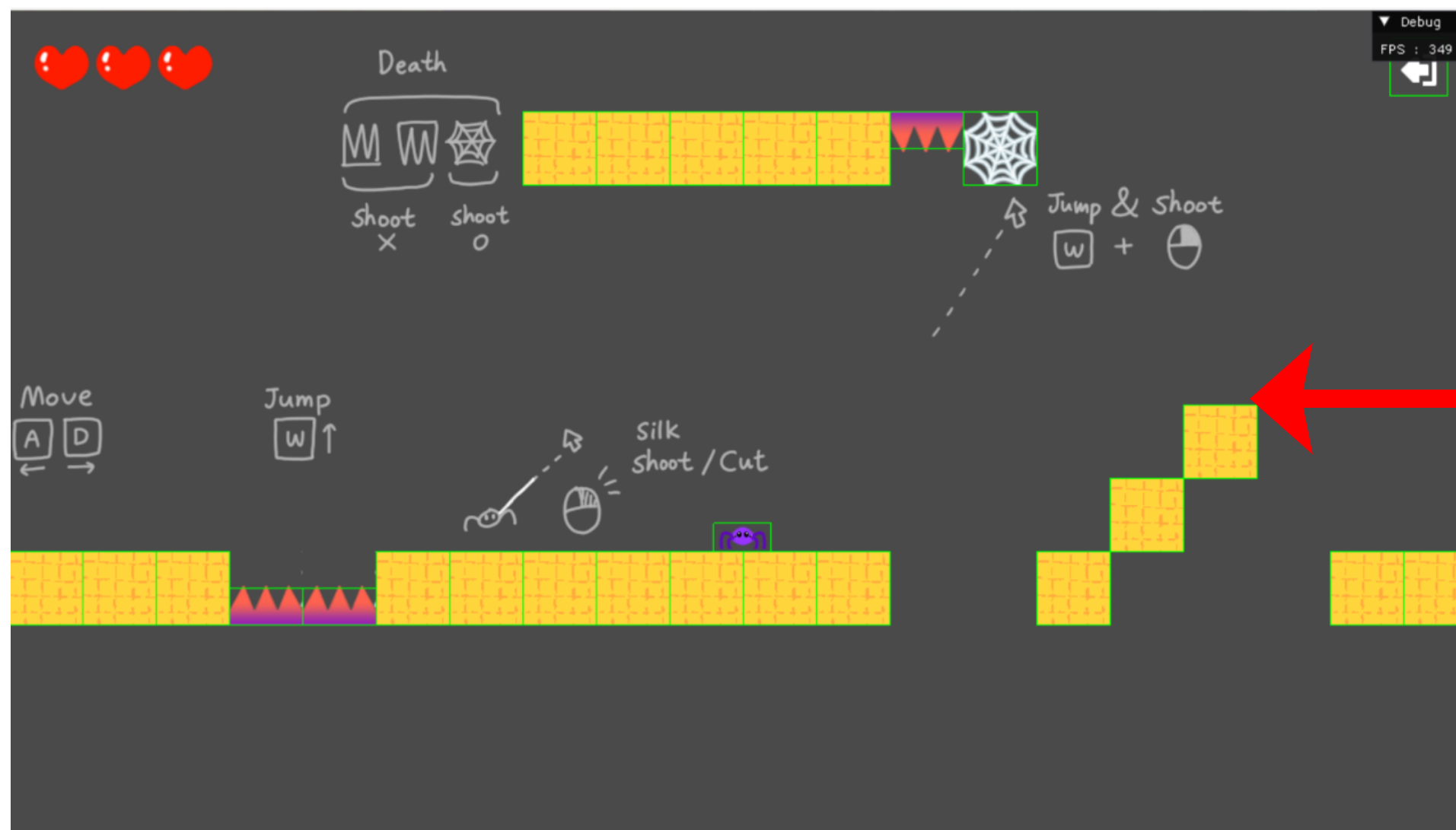
Editing 모드(게임 시작 전)



Game 모드 (설치된 타일이 그대로 적용됨)



게임 시작 버튼을 누를 시 StartGame 메소드를 호출하여  
각 타일의 특성에 따라 적절한 설정을 마친 후,  
Enemy와 Player 객체를 활성화



Camera 뷰 또한 FreeMode에서 FollowMode로 전환해  
플레이어를 따라 움직이도록 구현.  
부드러운 움직임 연출을 위해 선형보간을 이용함.

StartGame 메소드 일부

```
for (EditTileObject* editTile : editTileObjects)
{
    if (editTile->GetTileType() == NO_EDIT) //에디팅 불가 지역
    {
        editTile->GetInstanceData()->curFrame = { 0, 0 }; //빈 타일로 바꿔준 후
        UpdateInstanceData();
        editTile->SetActive(false);
    }
    else if (editTile->GetTileType() == CAN_EDIT && !editTile->GetIsEdited()) //
    {
        editTile->SetActive(false);
    }
    else if (editTile->GetTileType() == CAN_EDIT && editTile->GetIsEdited()) //
    {
        editTile->SetTileType(RAGULAR); //에디팅 된 타일 ->일반타일 붙임
    }
    else if (editTile->GetTileType() == PLAYER) //플레이어 타일
    {
        editTile->GetInstanceData()->curFrame = { 0, 0 };
        UpdateInstanceData();
        editTile->SetActive(false);
        player->SetActive(true); //플레이어 켜주기
    }
    else if (editTile->GetTileType() / 1000 == DeadObjectType::ENEMY) //적 타일
    {
        editTile->GetInstanceData()->curFrame = { 0, 0 };
        UpdateInstanceData();
        editTile->SetActive(false);
    }
    else if (editTile->GetTileType() == UP_THORN)
    {
        Vector2 tileSize = editTile->Size() * Vector2(1, 0.5f);
        Vector2 offset = { 0, -tileSize.x * 0.25f }; //콜라이더 크기 변경해주기
        editTile->Translate(offset);
        editTile->UpdateWorld();
        editTile->UpdateSize(tileSize);
    }
    else if (editTile->GetTileType() == DOWN_THORN)
    {
        Vector2 tileSize = editTile->Size() * Vector2(1, 0.5f);
        Vector2 offset = { 0, +tileSize.x * 0.25f }; //콜라이더 크기 변경해주기
        editTile->Translate(offset);
        editTile->UpdateWorld();
        editTile->UpdateSize(tileSize);
    }
}

isEditing = false;

//적 객체들 활성화
EnemyManager::Get()->TurnOnEnemyActivity();

CAM->SetTarget(player);
player->Drop();
```

# SpiderObject

## 플레이어와 적에 적용되는 물리법칙

플레이어와 적은 같은 물리법칙이 적용되므로  
중력을 적용시키는 RigidbodyObject와, 진자운동을 적용시키는  
SpiderPhysics 클래스를 부모로 두고 이를 상속받도록 함

```
void SpiderPhysics::PendulumMove()
{
    // 1. 현재 줄의 방향 벡터 구하기
    Vector2 ropeVec = pos - attachPos; // 현재 거미 위치 - 부착 위치 => 방향: 부착->거미
    Vector2 ropeDir = ropeVec.Normalized(); // == 줄이 향하는 방향(정규화)

    // 2. 중력 힘 설정 (항상 아래로 작용)
    Vector2 gravity = Vector2(0, -1) * MASS;

    // 3. "접선 방향 힘(Tangential Force)" 구하기
    // 원리: 중력(Gravity)에서 줄을 당기는 힘(Tension)을 제거하면 회전하려는 힘만 남음

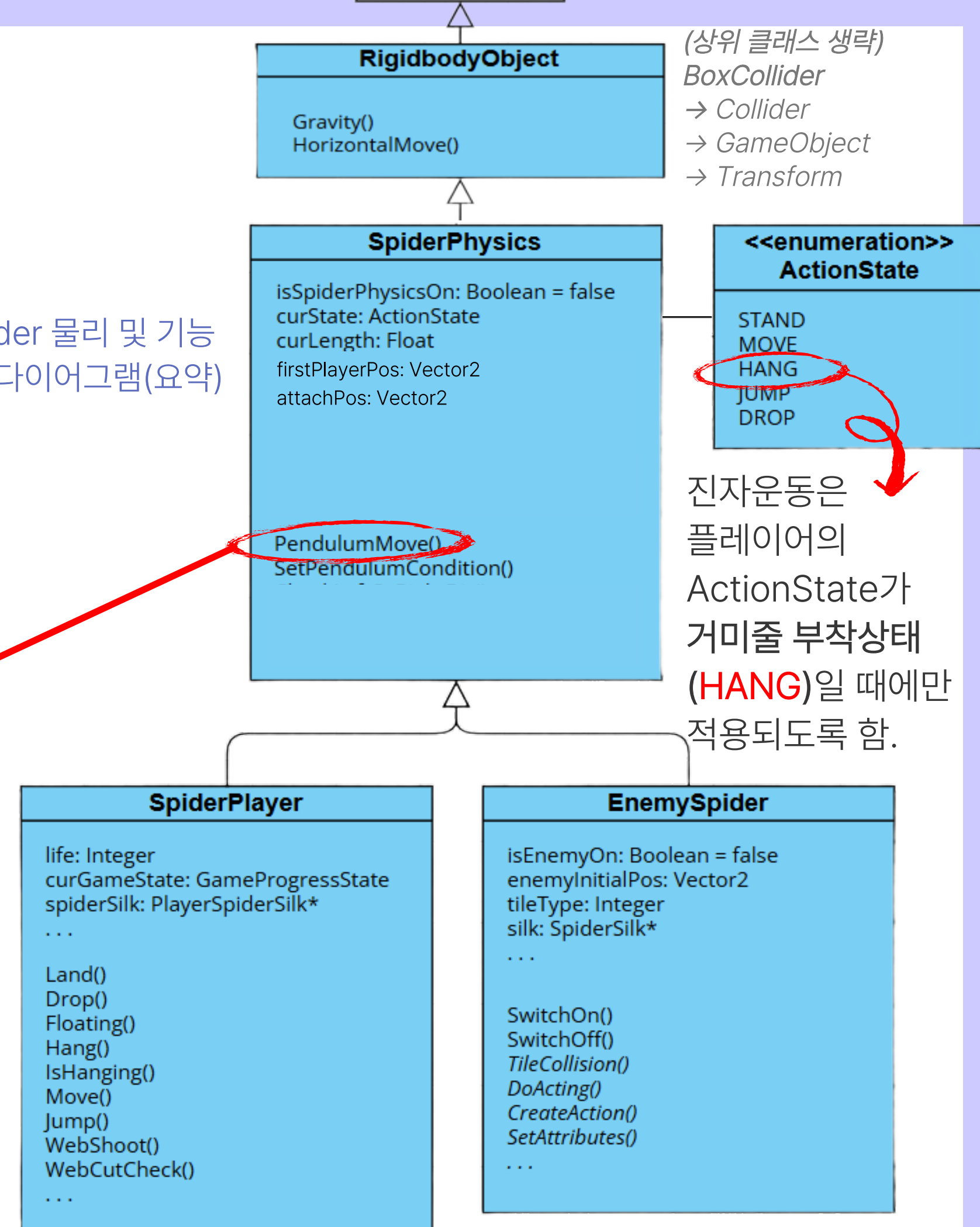
    // 3-1. 중력을 줄 방향에 투영(Projection) -> 줄을 당기는 힘 성분
    float tensionScalar = Vector2::Dot(gravity, ropeDir);
    Vector2 tensionForce = ropeDir * tensionScalar;

    // 3-2. 중력 - 줄 당기는 힘 = 순수하게 진자 운동을 시키는 힘 (접선력)
    Vector2 tangentialForce = gravity - tensionForce;

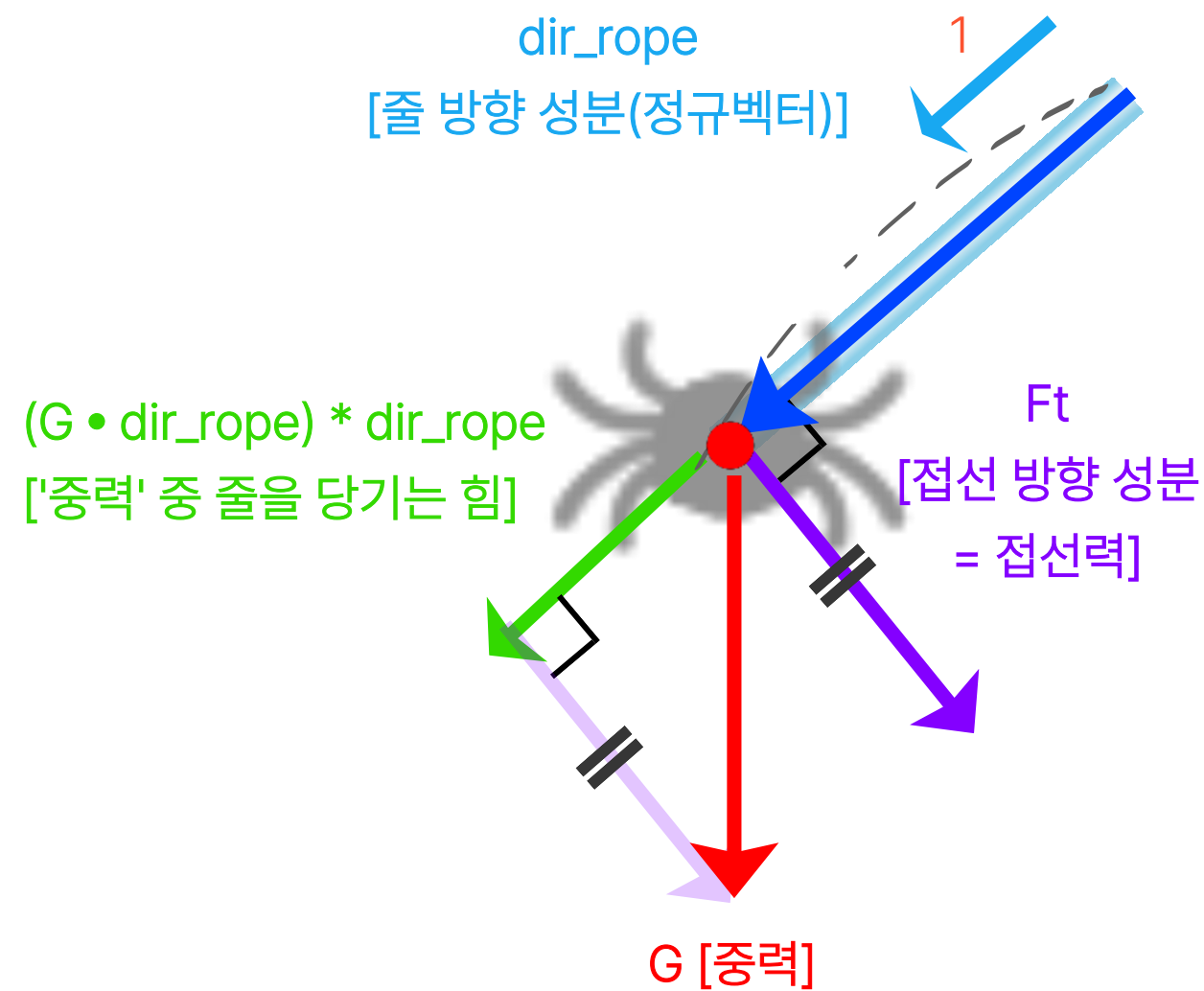
    // 4. 가속도 적용 (F = ma)
    velocity += tangentialForce * DELTA;
}
```

SpiderPhysics 내  
PendulumMove 메소드

Spider 물리 및 기능  
클래스 다이어그램(요약)







## (1) 중력 벡터 분해 (Vector Decomposition)

1. 매 프레임 중력( $G$ )을 '줄 방향 성분'과 '접선 방향 성분'으로 분해하여 물리 연산 수행
2. **접선력( $F_t$ )**: 중력에서 줄의 장력을 제외한, 실제 진자 운동을 일으키는 힘
3. 공식:  $F_t = G - (G \cdot dir\_rope) * dir\_rope$

## (2) 위치 보정 (Position Correction)

1. 연산 오차 방지를 위해 매 프레임 거미의 위치를 부착점 (Pivot)으로부터 줄의 길이( $L$ )만큼 떨어진 위치로 재설정

## (3) 속도 제약 조건 (Velocity Constraint)

1. 운동 에너지 보존과 궤도 안정을 위해 속도 벡터를 보정
2. 현재 속도( $V$ )에서 줄의 방향으로 향하는 성분( $V_{radial}$ )을 제거하여, 오직 접선 방향의 속도만 유지되도록 함
3. 공식:  

$$Velo\_new = Velo\_old - (Velo\_old \cdot dir\_rope) * dir\_rope$$

```
void SpiderPhysics::Update()
{
    if (!isSpiderPhysicsOn) //hang 상태가 아닌 경우엔 Rigidbody의 물리법칙을 따르도록
    {
        RigidbodyObject::Update();
    }
    else //hang 상태의 경우
    {
        // 1. 속도만큼 이동
        pos += velocity * DELTA;

        Vector2 ropeVec = pos - attachPos;
        Vector2 ropeDir = ropeVec.Normalized();

        // 2. 위치 강제 보정 (줄 길이 유지)
        // 거미가 줄보다 멀어지거나 가까워지면 강제로 줄 길이만큼 떨어진 곳으로 위치시킴
        pos = attachPos + ropeDir * curLength;

        // 3. 속도 벡터 보정 (Radial Velocity Removal)
        // 위치를 강제로 옮겼어도 Velocity가 줄 방향(바깥쪽)을 향하고 있으면
        // 다음 프레임에 또 밖으로 튀어나가려고 해서 불안정하게 흔들릴 수 있음.
        // 따라서 Velocity에서 '줄 방향 성분'을 0으로 잘라버려야 깔끔하게 됨.

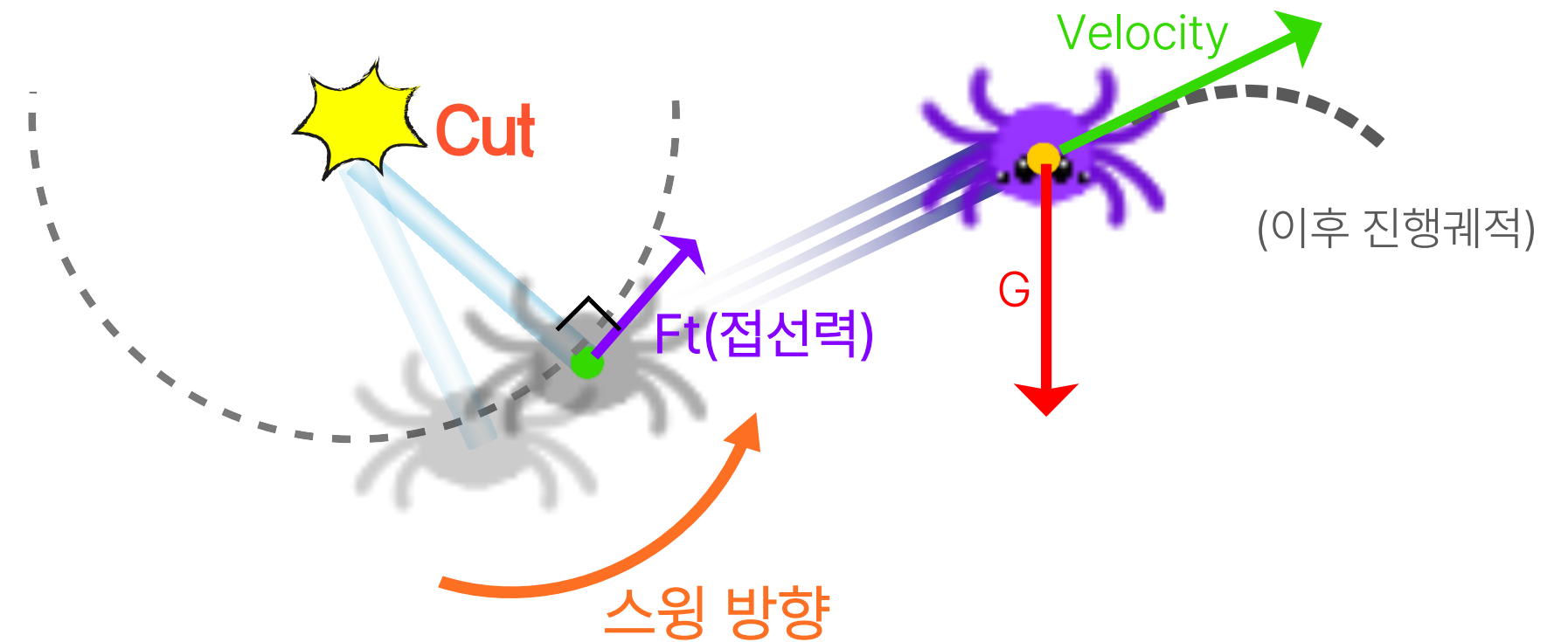
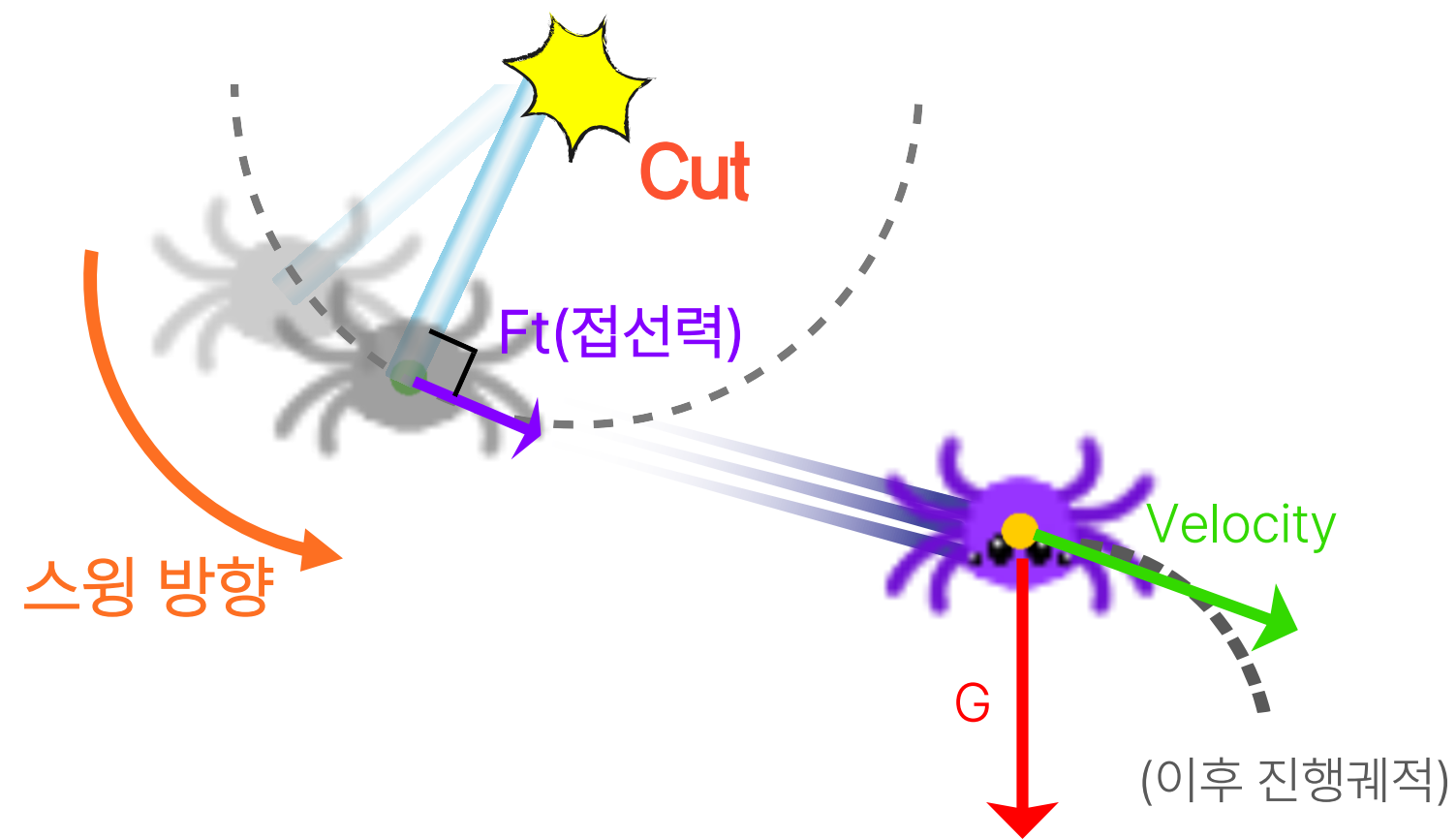
        float radialVel = Vector2::Dot(velocity, ropeDir); // 속도 중 '줄 방향 성분' 크기
        velocity -= ropeDir * radialVel; // 순수 접선 속도만 남김

        UpdateWorld();
    }
}
```

Spiderphysics 클래스의 Update() 메소드

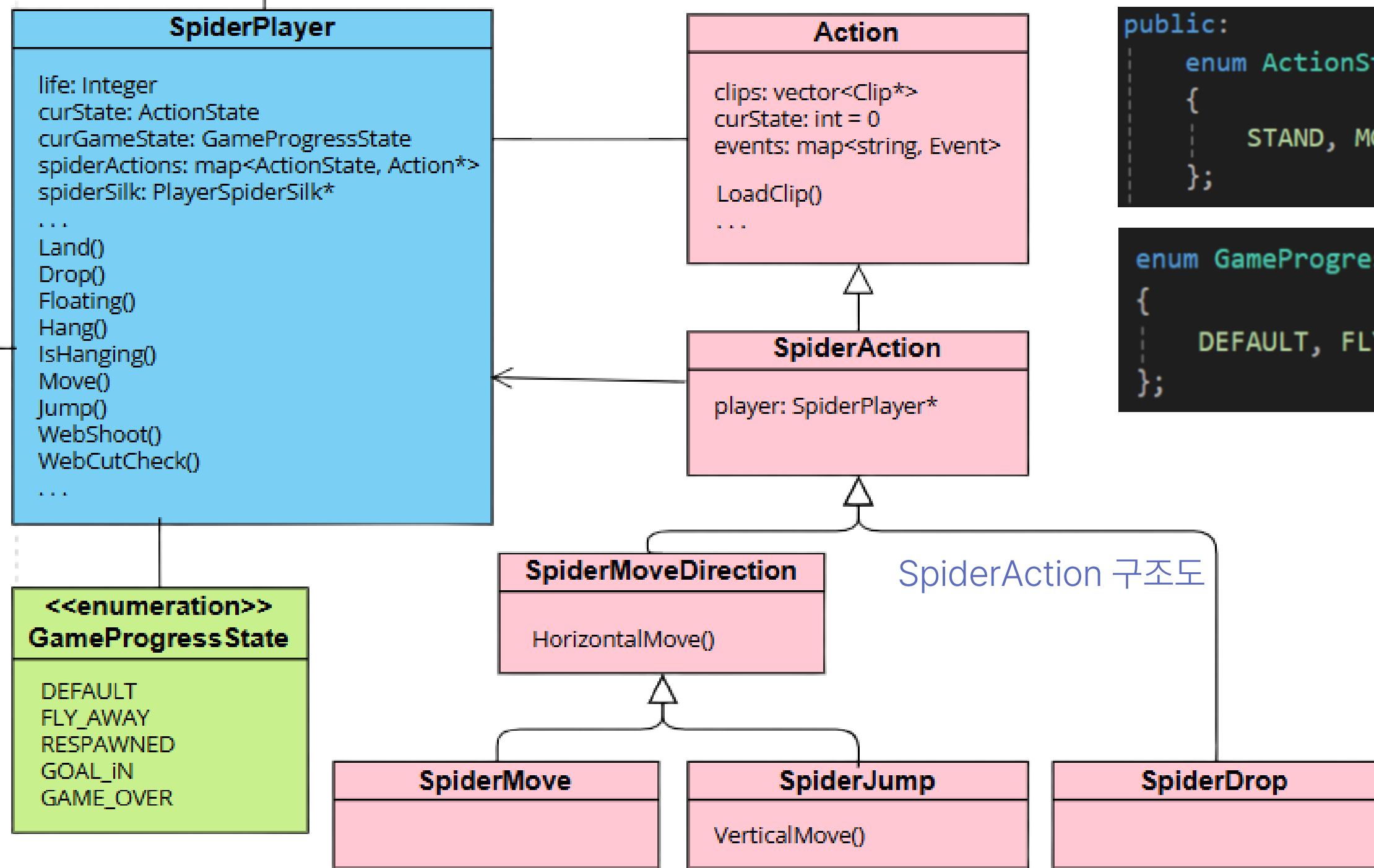
#### (4) 관성 보존 (Momentum Conservation)

1. 별도의 인위적인 힘 적용 없이 물리 엔진의 연산만으로 운동량 보존 법칙을 구현
2. 줄이 끊어지는 순간(Cut), 매달려 있던 상태의 접선 속도(Tangential Velocity)가 그대로 유지되어 낙하 운동으로 전환됨



#### (5) 자연스러운 상태 전이 (Seamless Transition)

1. HANG 상태의 물리 연산이 DROP 상태의 중력 연산으로 단절 없이 이어지도록 설계
2. 스윙의 방향과 속도에 따라 다양한 포물선 궤적을 그리며 날아가도록 구현



```

public:
    enum ActionState
    {
        STAND, MOVE, HANG, JUMP, DROP
    };
  
```

```

enum GameProgressState //기본, 끊고 날아갈 때, 리스폰된 상황, 골인, 게임오버
{
    DEFAULT, FLY_AWAY, RESPAWNED, GOAL_IN, GAME_OVER
};
  
```

ActionState와 GameProgressState

플레이어의 게임 진행 상태를 나타내는  
enum 형식의 GameProgressState를 이용해  
리스폰, 골인 여부, 게임오버 여부 등을 관리



좌측 상단부터 우측 하단 방향으로  
STAND, MOVE, HANG, JUMP, DROP 상태 시 재생되는 클립들

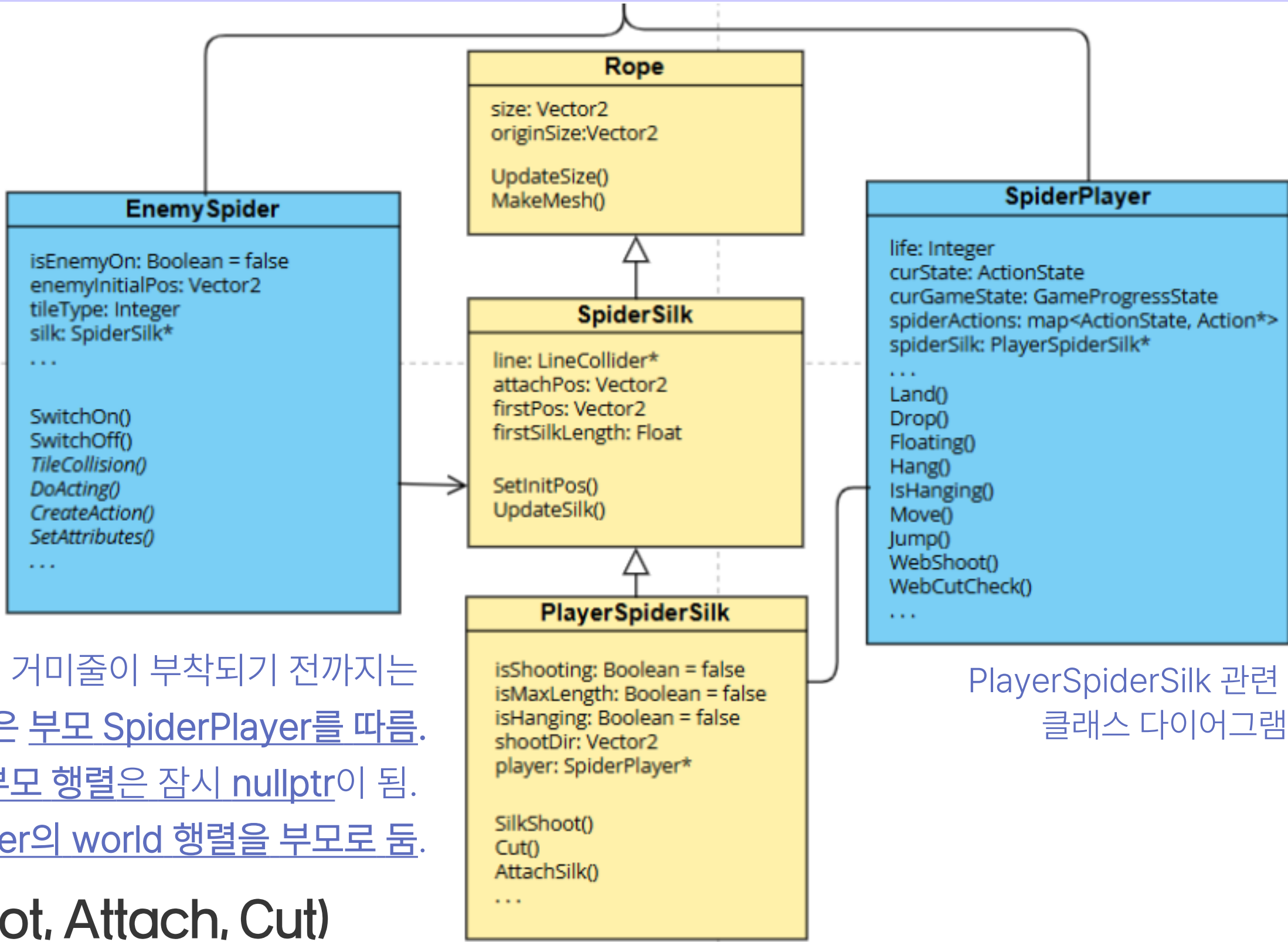
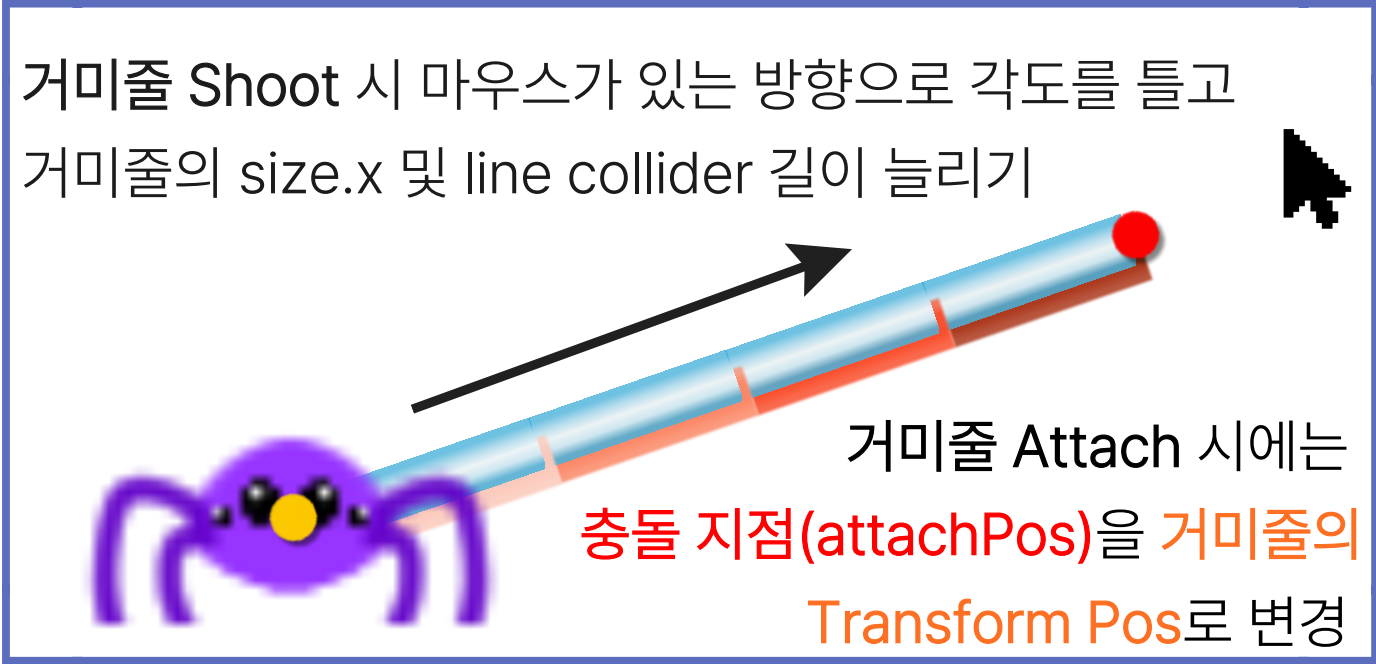
## SpiderPlayer의 행동객체화 및 게임 진행 상태 표시

플레이어는 현재 취한 액션 상태에 따라 enum으로 ActionState를 나눔.  
(STAND, MOVE, HANG, JUMP, DROP)

행동 객체화를 통해 상태 별로 애니메이션 클립을 저장하여 현재 상태  
(curState)에 맞게 재생하고, 이에 해당하는 기능을 수행하도록 함.



거미줄 기본이미지.  
**Transform Pos**는 좌측 중앙(노란 점)으로 설정됨.



PlayerSpiderSilk 관련  
클래스 다이어그램

### PlayerSpiderSilk 애니메이션 및 기능 (Shoot, Attach, Cut)

SpiderPlayer가 지니고 있는 거미줄인 PlayerSpiderSilk는 마우스 우클릭 시 현재 쏘는 중인지(isShooting), 매달렸는지(isHanging) 등을 판단해 거미줄의 길이인 size.x를 실시간으로 업데이트 하는 식으로 구현함.

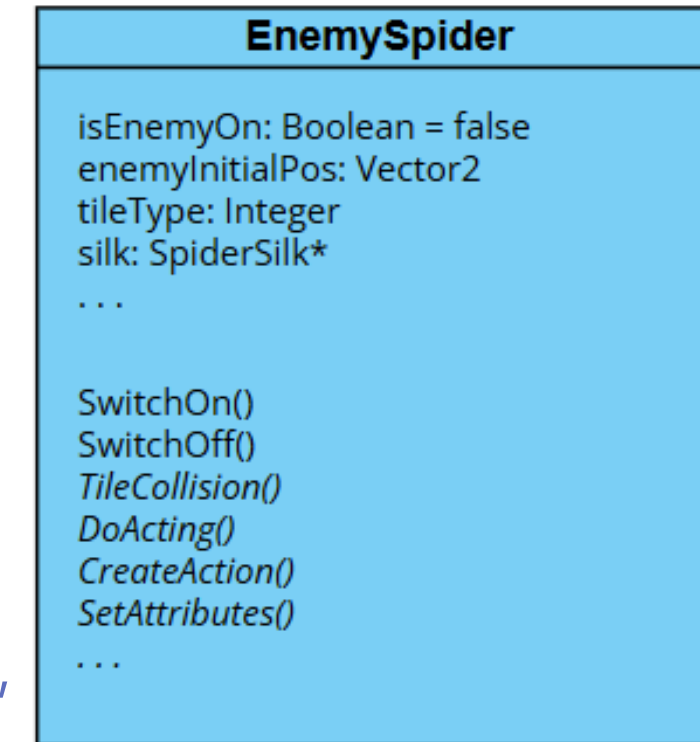
(이때 텍스처가 단순하게 늘어지는 것을 방지하기 위해 UV Sampler에서 AddressU를 Mirror 모드로 설정하여 자연스러운 타일링(Tiling) 효과를 구현함)

# EnemySpider 종류 구분 및 EnemyManager

싱글톤으로 구현된 EnemyManager에서 EnemySpider를 관리.  
이동(Move) 유형, 점프(Jump) 유형, 매달린(Hang) 유형이 존재함. (EnemySpider 상속)  
SpiderGame에서 맵 로드 과정에서 적 객체 초기화 시  
순수가상함수로 구현한 SetAttributes() 메소드로 각 적의 종류별로 속성값을 저장  
(Move 유형은 이동 거리(칸 수)와 속도 배율, Jump 유형은 점프 배율, Hang 유형은 줄 길이 배율)

```
private:
    virtual void DoActing() = 0; //jump, move, hang
    virtual void CreateActions() = 0;
    virtual void SetAttributes() = 0;
```

Enemy 종류  
(Move, Jump, Hang)



적 생성 시 tileType 숫자를 4자리로 받음.  
앞 두 자리는 거미 유형(20, 21, 22)을 의미하며,  
뒤 두 자리는 각 적의 고유 속성값을 의미.

```
void MoveEnemySpider::SetAttributes()
{
    /** tileType 정보 예시 **/
    // ex) tileType = 2143
    // 21: MoveEnemy
    // 4: 이동 범위(칸 수)
    // 3: 이동 속도 배율 (지정된 속도의 3배 빠르기로 이동)
    int remainNum = tileType % 100;
```

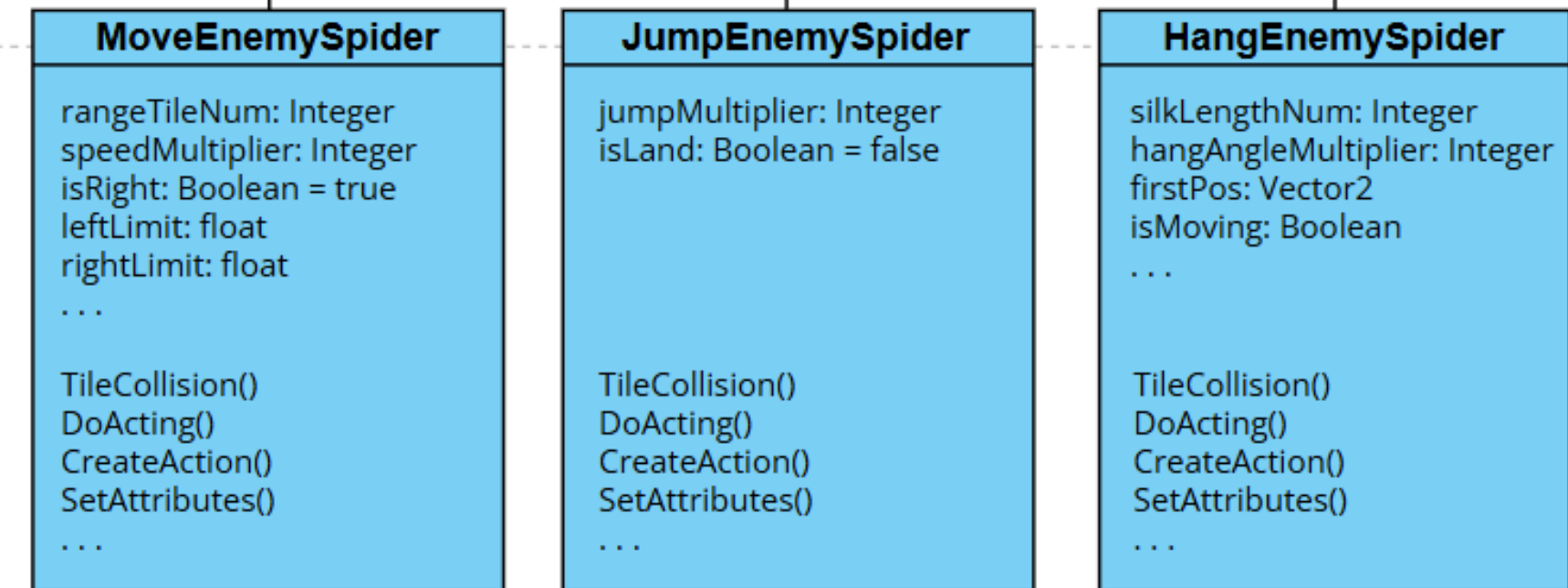
```
void JumpEnemySpider::SetAttributes()
{
    /** tileType 정보 예시 **/
    // ex) tileType = 2060
    // 20: JumpEnemy
    // 60: 점프 파워 배율 (지정된 파워(1

    int remainNum = tileType % 100;
    jumpMultiplier = remainNum;
}
```

```
void HangEnemySpider::SetAttributes()
{
    /** tileType 정보 예시 **/
    // ex) tileType = 2213
    // 22: HangEnemy
    // 1: 줄 길이(칸 수)
    // 3: 흔들리는 각도 배율 (0은 흔들리지 않음)

    int remainNum = tileType % 100;

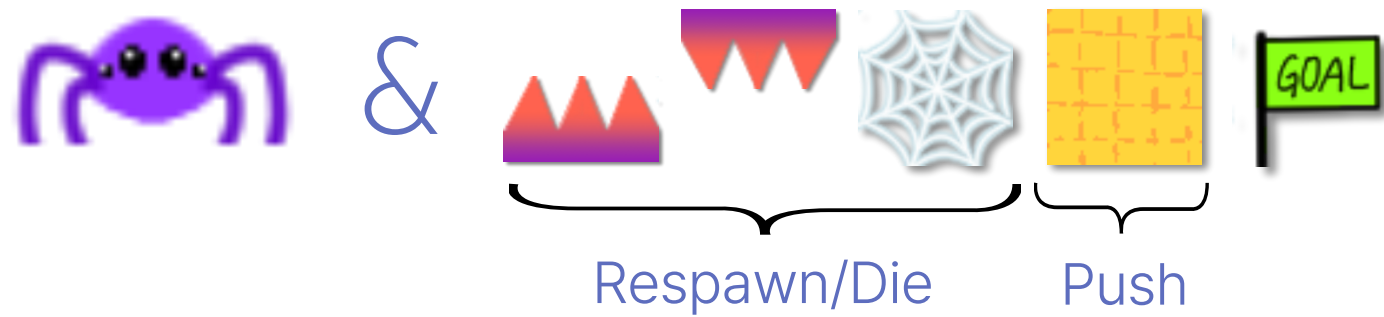
    silkLengthNum = remainNum / 10;
    hangAngleMultiplier = remainNum % 10;
}
```



EnemySpider 클래스 다이어그램

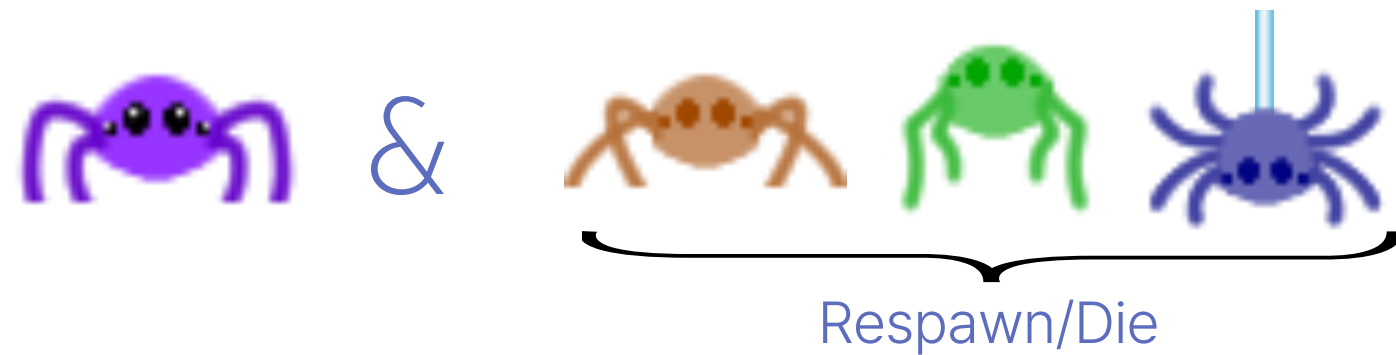
### 1) 플레이어 & 맵 타일 충돌

- [SpiderGame의 PlayerCollision 메소드](#)로 처리



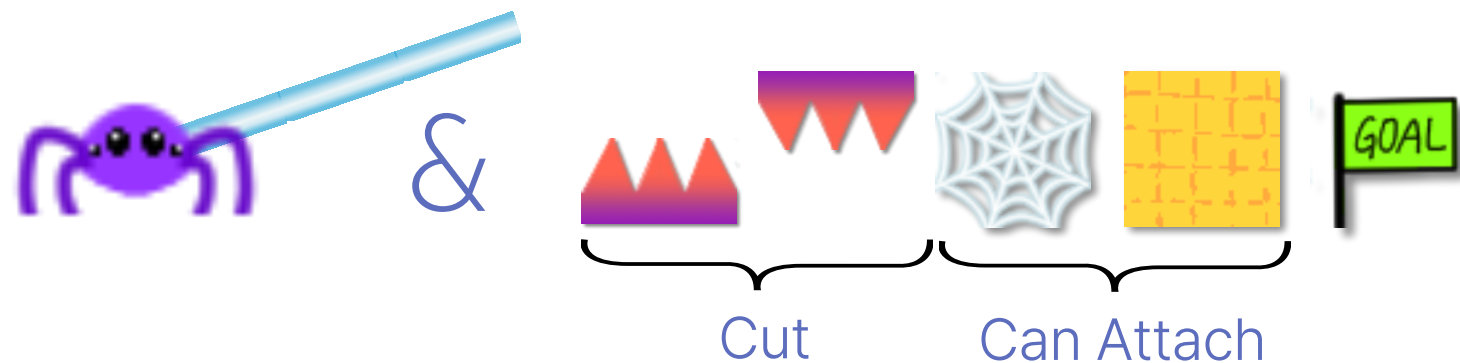
### 2) 플레이어 & 적 충돌

- [EnemyManager에서 CheckPlayerCollision 메소드](#)로 처리



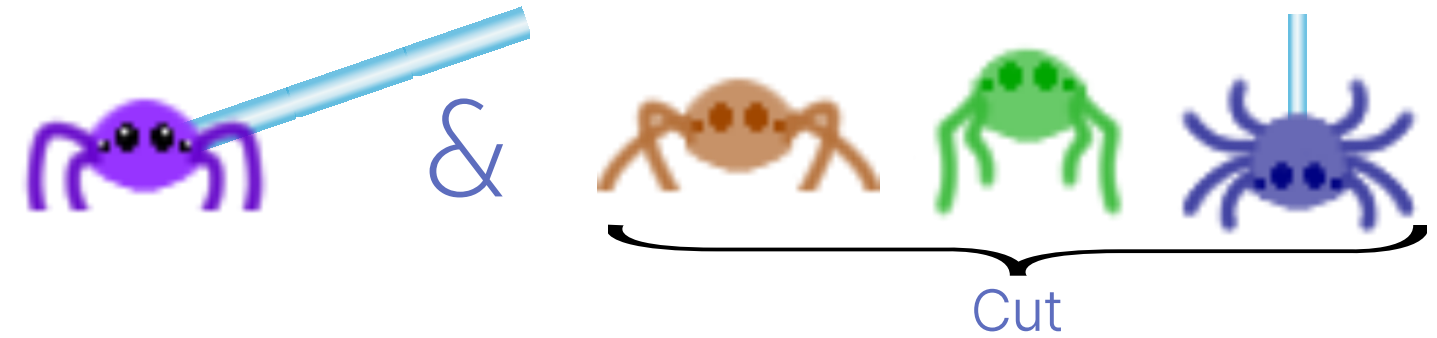
### 3) 플레이어 실 & 맵 타일 충돌

- [EditTileObject에서](#) 부착 가능 범위에 충돌하면  
순수가상함수로 구현된 [SilkCollision 메소드](#) 호출
- 만약 부착 상태일 때 다른 벽돌과 충돌하면 줄이 끊어지도록 예외처리



### 4) 적 & 플레이어 실 충돌

- SpiderGame에서 [EnemyManager의 CollisionMapAndSilkCheck 메소드](#) 호출



### 5) 적 & 맵 타일 충돌

- SpiderGame에서 [EnemyManager의 CollisionMapAndSilkCheck 메소드](#) 호출
- 충돌 시 EnemySpider의 순수가상함수 [TileCollision 메소드](#)를 호출해  
적 종류별로 필수적인 충돌 처리만 구현 (Land 처리 등)



Spider, SpiderSilk, Enemy, Tile의 요소별 충돌처리



```

else //게임 종일 때
{
    CheckPlayerInMapArea();
    player->Update();

    //플레이어 & 적 충돌
    EnemyManager::Get()->Update();

    for (EditTileObject* editTile : editTileObjects)
    {
        //플레이어 & 타일 충돌
        PlayerCollision(editTile);

        //플레이어 줄 & 타일 충돌
        editTile->Update();

        //적 & 플레이어 줄/타일 충돌
        EnemyManager::Get()->CollisionMapAndSilkCheck(editTile);
    }
}

```

SpiderGame의 Update 메소드 일부

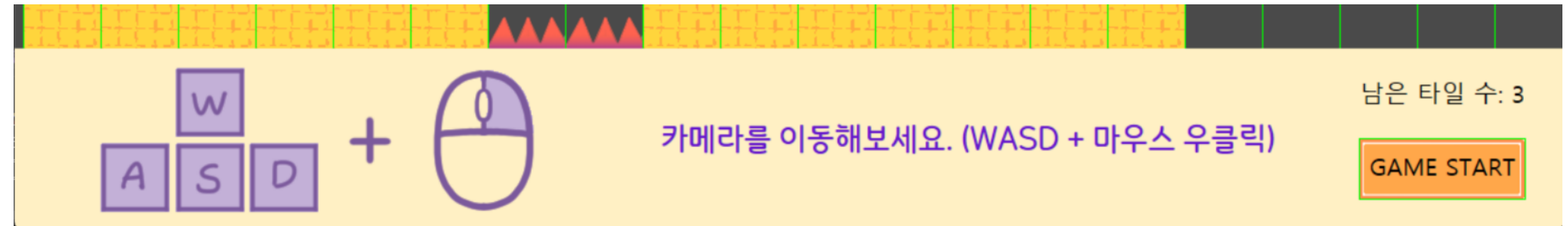
```

void SpiderGame::PlayerCollision(EditTileObject* editTile)
{
    //editTile 타입에 따라 충돌 체크하기
    switch (editTile->GetTileType())
    {
        case RAGULAR:
            ((BasicTile*)editTile)->CollisionCheck();
            break;
        case DOWN_THORN: case UP_THORN: case COBWEB:
            ((ObstacleTile*)editTile)->CollisionCheck();
            break;
        case GOAL:
            ((GoalTile*)editTile)->CollisionCheck();
            break;
    }
}

```

SpiderGame의 PlayerCollision 메소드 일부

# UI 및 기타 기능

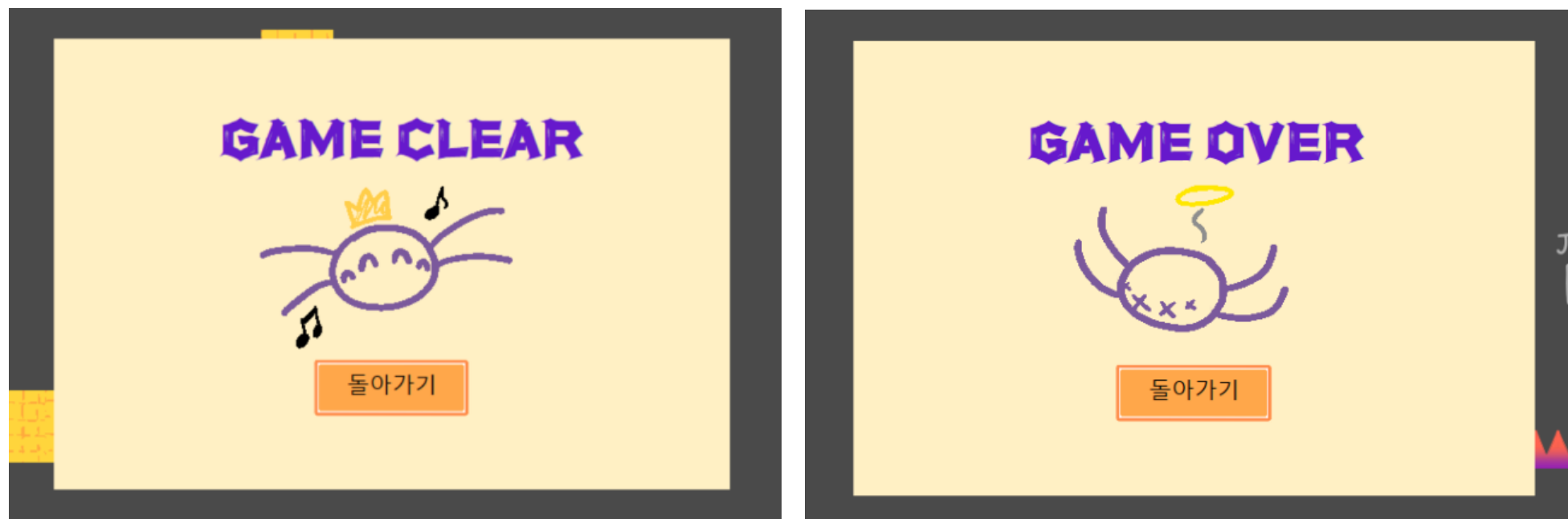


튜토리얼 창을 생성하는 TutorialUIPanel(UIPanel을 상속)

## UI Manager를 이용한 창 관리

UIPanel를 부모로 두어 필요한 UI 창을 관리.

튜토리얼 안내창, 게임 종료 시 확인창 등이 앞에 떠 있는 경우에는 해당 창 내부를 클릭 시 뒷배경의 버튼, 에디팅 기능이 인식되지 않도록 예외처리 (IsContain 메소드로 구현)



게임 결과를 출력하는 GameResultUIPanel(UIPanel을 상속)

```
bool UIManager::IsContain(string strName, Vector2 curPos)
{
    //해당 패널이 꺼져있으면 체크안함
    if (!panels[strName]->IsOn()) return false;

    Vector2 panelHalfSize = panels[strName]->GetSize() * 0.5f;
    Vector2 panelPos = panels[strName]->GetPos();

    //UI 바깥이면 false
    if (curPos.x < panelPos.x - panelHalfSize.x) return false;
    if (curPos.x > panelPos.x + panelHalfSize.x) return false;
    if (curPos.y < panelPos.y - panelHalfSize.y) return false;
    if (curPos.y > panelPos.y + panelHalfSize.y) return false;

    //아니면 true
    return true;
}
```

UIManager 클래스 내의 IsContain 메소드

# Tutorial 내용 자동 전환 구현

TutorialUI의 경우 특정 조건을 충족할 때마다 다음 내용과 이미지를 출력하도록 구현



Tutorial 창 출력 모습

```
if (game->GetIsEditing())
{
    switch (tutorialUI->GetCurPage())
    {
        case 0:
            //카메라가 오른쪽으로 일정 수치 이상 이동하면
            if (CAM->GetPos().x > 150.0f)
                tutorialUI->NextPage(); //다음 페이지로
            break;
        case 1:
            //모든 타일을 다 놓았다면
            if (game->GetLeftEditTileNum() == 0)
                tutorialUI->NextPage(); //다음 페이지로
            break;
        case 2:
            break;
    }

    gameStartButton->Update();
}
```

TutorialScene의 Update 메소드 일부

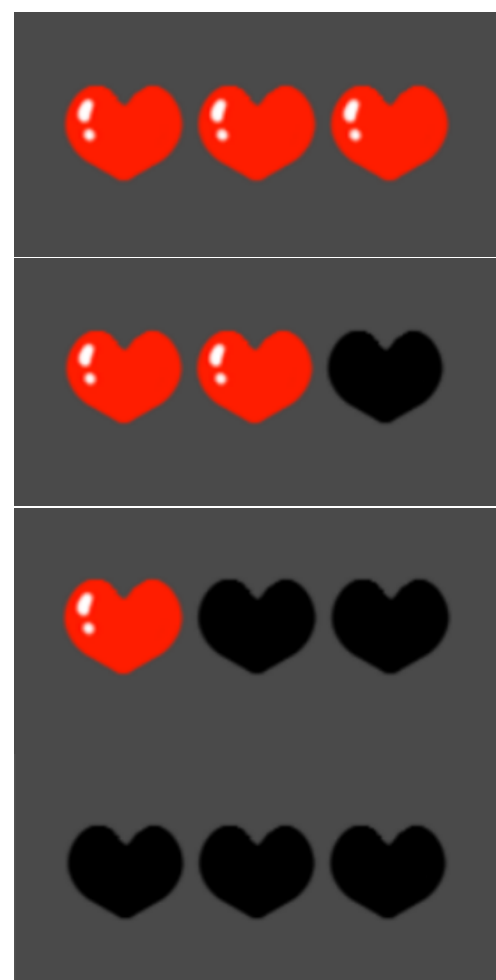


```
void SpiderPlayer::DamageEffect(float curTime)
{
    //빨간색 점멸 (0.2초 간격)
    blinkTime += DELTA;

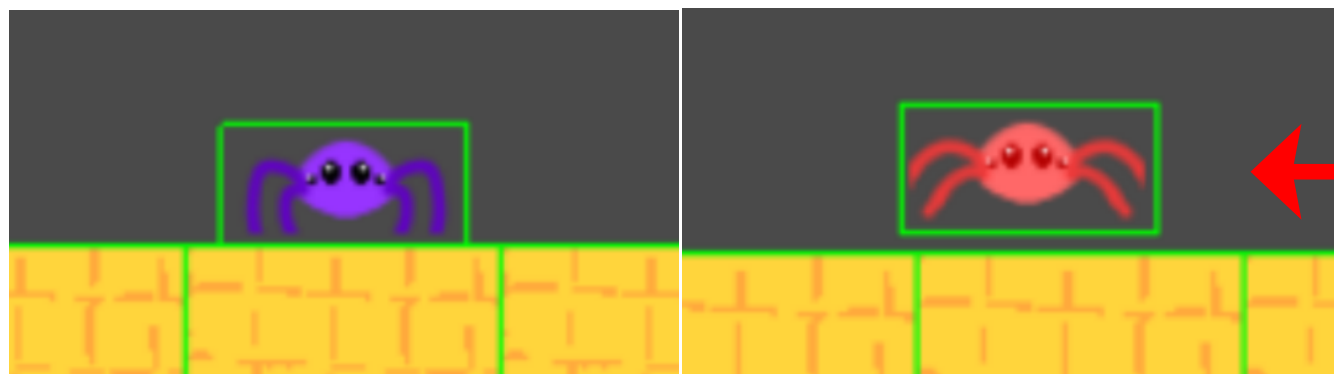
    if (blinkTime >= BLINK_TIME)
    {
        if (intValueBuffer->Get()[SWITCH_SLOT] == OFF)
            intValueBuffer->Get()[SWITCH_SLOT] = ON;
        else
            intValueBuffer->Get()[SWITCH_SLOT] = OFF;

        blinkTime -= BLINK_TIME;
    }
}
```

플레이어 피격 시 점멸효과를 주는  
DamageEffect 메소드



Slider.hlsl을 이용한  
목숨 표시 UI



피격 시 점멸 모습

```
//ValueBuffer (점멸 스위치, 점멸 시 색상)
cbuffer ValueBuffer : register(b2)
{
    int isSwitchOn;
}

cbuffer ValueBuffer : register(b3)
{
    float4 overlayColor;
}
```

Player.hlsl의 ValueBuffer

```
float4 PS(PixelInput input) : SV_TARGET
{
    float2 uv = input.uv + curFrame / maxFrame;

    float4 baseColor = map.Sample(samp, uv);

    if (baseColor.r == removeColor.r &&
        baseColor.g == removeColor.g &&
        baseColor.b == removeColor.b)
        baseColor.a = 0.0f;

    if (isSwitchOn == 1) //오버레이 기능 켜지면 (피격 시)
        baseColor = Grayscale2(baseColor) + overlayColor;

    return baseColor * tintColor;
}
```

0은 OFF, 1은 ON

Player.hlsl의 PS 연산

## Shader를 이용한 기능 (남은 목숨 표시, 플레이어 피격 시 붉은색 점멸)

목숨의 경우 Slider 셰이더를 이용해 이미지 가로 비율에 따라 UV의 x 값을 조절.  
피격 시 점멸은 리스폰 시 일정 시간동안 미리 지정한 색으로 오버레이 효과를 주어  
0.2초 간격으로 ON(1), OFF(0) 값을 번갈아가며 intValueBuffer에 전달하도록 구현

## 사운드, 이펙트 처리

FMOD 라이브러리를 활용하여  
싱글톤으로 Audio 객체를 생성해 각종 효과음, 배경음을 재생하도록 함

```
//시작화면 브금 & 버튼 효과음
Audio::Get()->Add("BGM", "Sounds/Start_bgm.wav", true);
Audio::Get()->Add("Button_StartScene", "Sounds/ButtonClick_Start.wav", false, false);

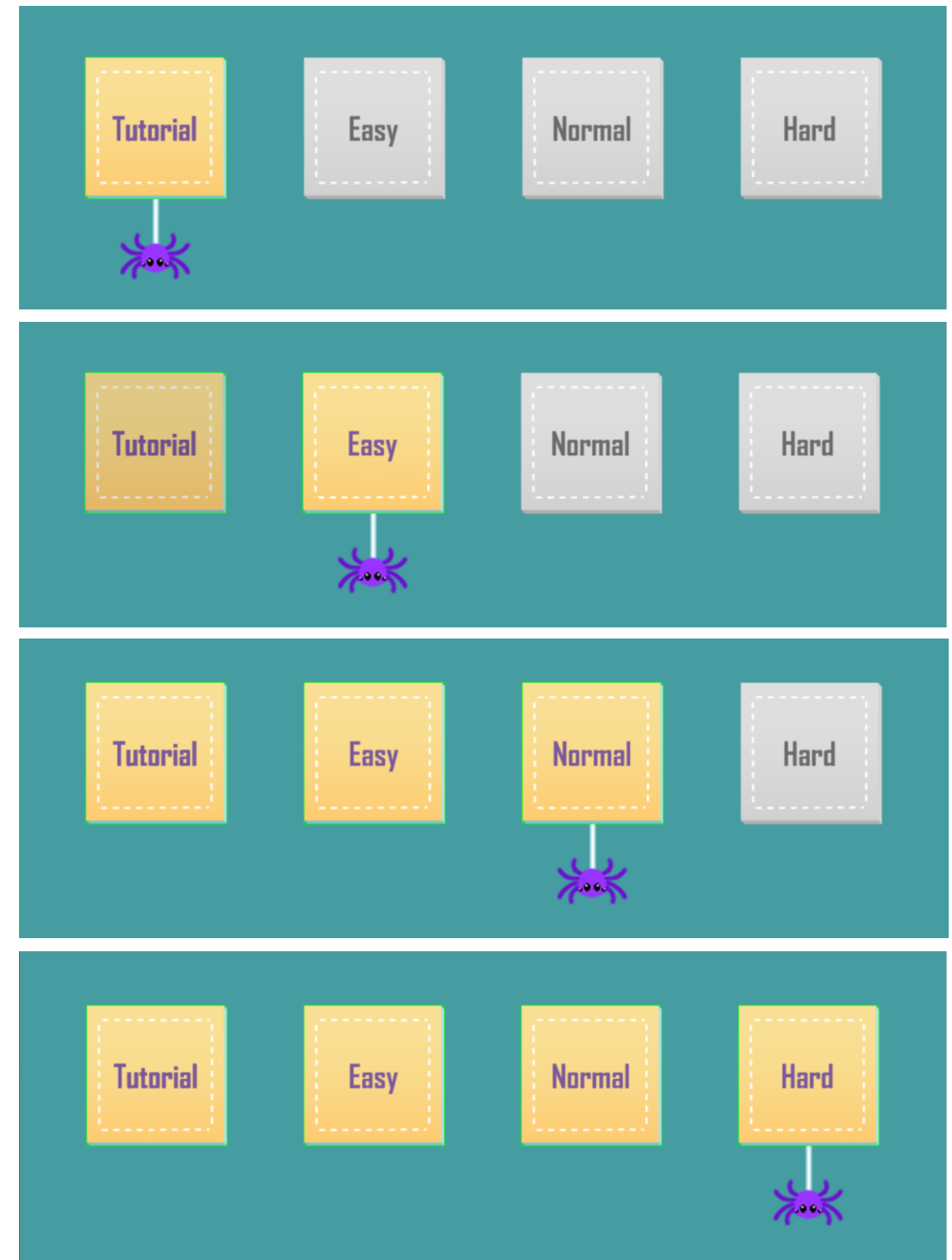
//플레이어 관련 효과음
Audio::Get()->Add("PlayerJump", "Sounds/PlayerJump.wav");
Audio::Get()->Add("SilkCut", "Sounds/SilkCut.wav");
Audio::Get()->Add("SilkShoot", "Sounds/SilkShoot.wav");
Audio::Get()->Add("SilkAttach", "Sounds/SilkAttach.wav");
Audio::Get()->Add("Damage", "Sounds/Damage.mp3");

Audio::Get()->Play("BGM", 0.3f);
```

시작화면 Scene 생성 시  
Audio 객체에 효과음, 배경음 추가

## 스테이지 클리어 정보 저장

현재 클리어 상태를 저장하여 다음 실행 시 다음으로 깨야하는 스테이지를 표시



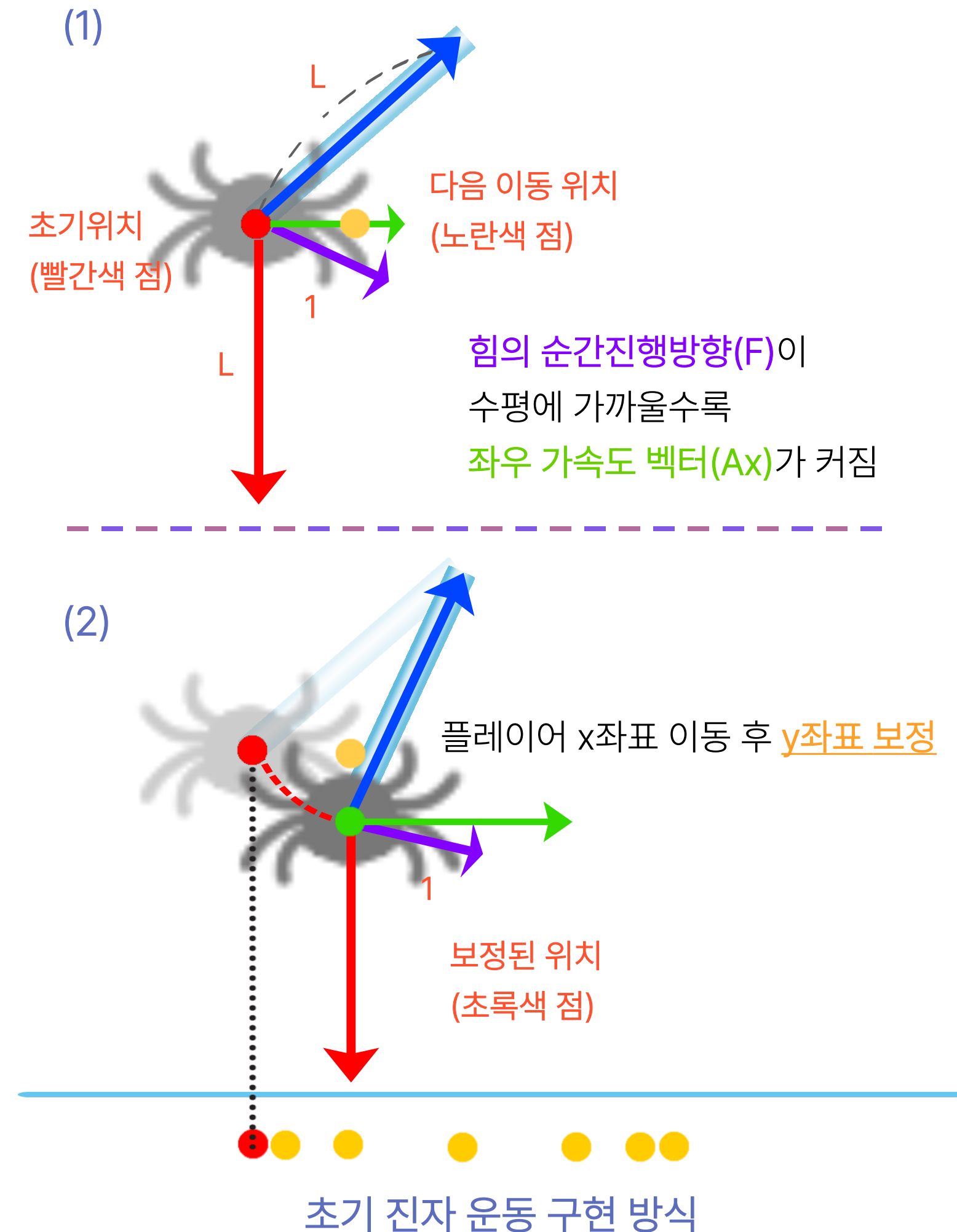
다음 스테이지 해금 모습

# Review

## [1. 트러블 슈팅 - 진자 운동 구현]

- 초기 접근: 거미의 진자 운동을 구현하기 위해 "초기 퍼텐셜 에너지 ( $E_0$ )"를 미리 계산해두고, 매 프레임 이동 방향 벡터를 '중력 + 장력'의 방향으로 구해 다음 위치의 x좌표로 이동시키는 방식으로 구현
- 발생한 문제: 진자가 최저점을 지날 때, 중력과 장력이 평형을 이루면서 가속도 벡터가 0이 되어 운동 방향이 소실되는 문제가 발생
- 임시 조치: CheckLeftOrRightDir 메소드로 거미가 현재 왼쪽에서 내려왔는지, 오른쪽에서 내려왔는지 상태를 인위적으로 체크하고 강제로 힘을 가하는 예외처리를 추가해야 했음.

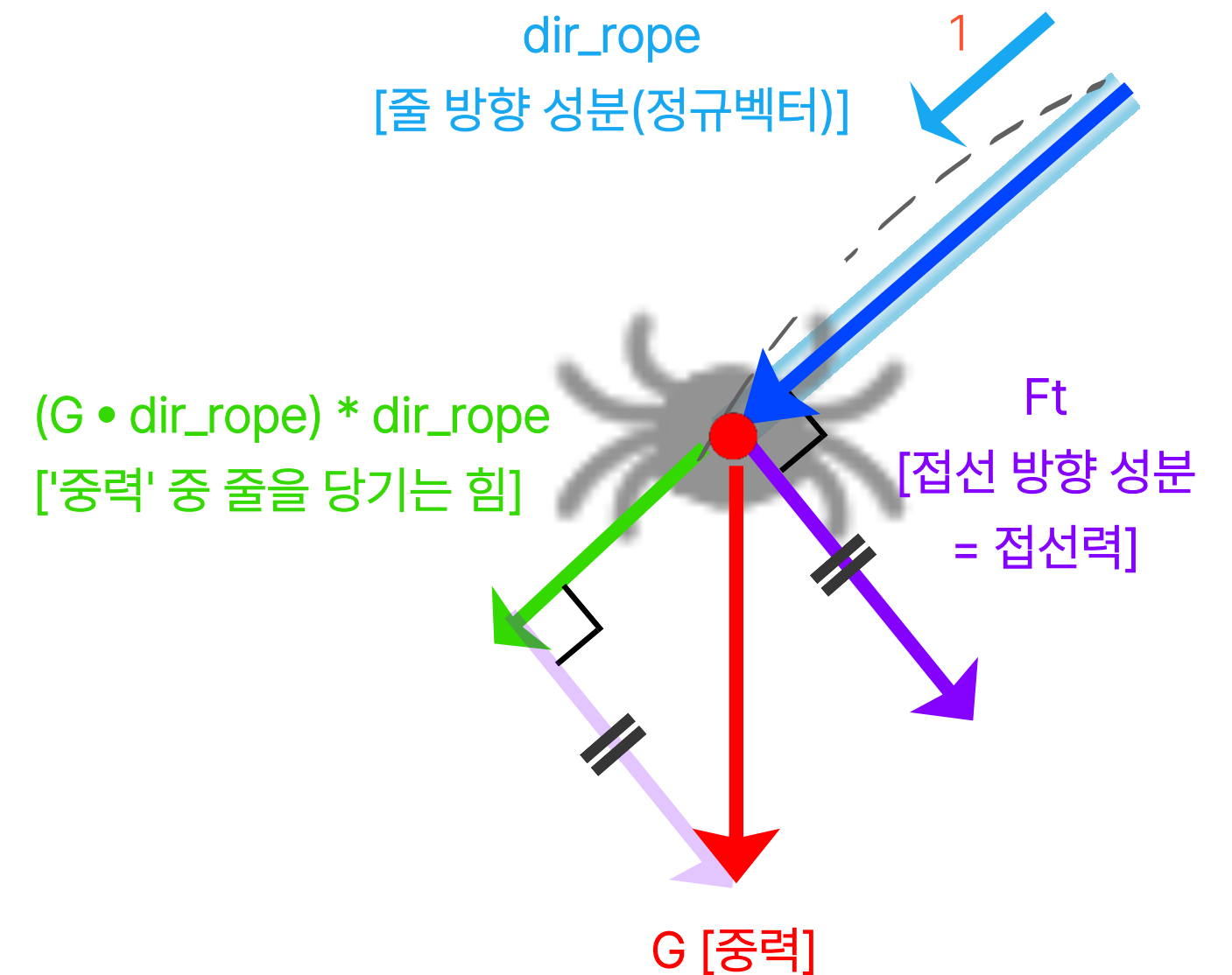
⇒ 이는 물리 법칙에 기반한 자연스러운 움직임이 아니라, 수치적인 벡터 합에 의존한 이동 방식이었기에 에너지 보존이 되지 않고, 궤도를 유지하기 위한 보정 코드가 계속 늘어나는 구조적 한계가 있었음



# Review

## [해결과정]

- \* 물리 엔진 재설계: 벡터 분해 기반의 물리 연산으로 로직 변경
- \* **접선력(Tangential Force)**: 중력을 '줄의 방향'과 '운동 방향'으로 분해하여, 실제 회전에 기여하는 힘만 정확히 추출해 가속도로 적용 ( $F = ma$ )
- \* 속도 제약: 매 프레임 속도 벡터에서 '줄 밖으로 나가려는 성분(Radial Component)'을 수학적으로 제거 (Dot Product 이용)
  - 결과: 복잡한 분기문 없이 단 하나의 물리 수식으로 최저점/최고점에서 발생하는 문제를 해결하고, 에너지 손실 없는 무한 진자 운동을 구현함



개선된 진자 운동 구현 방식



# Review

## [2. 설계 의사결정: 타일 시스템의 구조]

- 타일의 종류(Type)에 따라 생성, 게임 모드 변환 과정에서 타일 기능 변환 처리, 충돌 처리 등을 분기문(switch-case, if-else)으로 제어하는 절차적 방식을 사용했는데, 이는 객체지향 설계 원칙 중 **OCP(개방-폐쇄 원칙)**에 위배되는 형태임. 만약 추후 새로운 타일이 추가된다면 SpiderGame 클래스 내부를 수정해야 하는 강한 결합(Coupling)이 존재함을 인지함.

⇒ 하지만 해당 프로젝트는 타일의 종류가 고정적(일반지형, 장애물, 적 타일 등)이며, 기획 단계에서 추가적인 타일 확장이 없을 것이라 판단하였음. 따라서 과도한 추상화(Over-Engineering)를 피하고 YAGNI(You Aren't Gonna Need it) 원칙에 입각하여, 직관적인 구현을 통해 개발 속도와 가독성을 확보하는 것을 우선으로 하였음.

# Review



## [3. 그 외]

- 직접 게임을 기획해보고, Stage 배경을 제외한 나머지 디자인 리소스들을 직접 제작해보면서 게임 개발의 전반적인 과정에 대해 알게 되었음.

- Enemy를 생성할 때는 풀링 기법을 활용하고, 스테이지에서 스크린 바깥에 있는 EditTileObject 및 EnemySpider는 충돌처리 Update를 진행하지 않음으로써 게임 최적화를 진행할 수 있었음.



Start Game

Tutorial